

48-portfolio-project-series

□ Portfolio Project Series

Level: Advanced

Prasyarat: Modul 06 (Backend), Modul 11 (Frontend)

Estimasi Waktu: 4-5 minggu (1 project/minggu)

Output Akhir: 5 proyek nyata yang siap ditunjukkan ke recruiter

□ Deskripsi

Modul **Portfolio Project Series** adalah puncak dari kurikulum RPL AI. Berbeda dari modul sebelumnya yang bersifat tutorial, modul ini berisi **5 proyek mandiri** yang dikerjakan step-by-step. Setiap proyek dirancang untuk menjadi portfolio piece yang berdiri sendiri.

Kamu akan membangun: 1. **Landing Page** — landing page personal portfolio dengan HTML/CSS/Tailwind, deploy ke Vercel/Netlify 2. **CRUD API** — Bookshelf API dengan Express + TypeScript + Prisma + PostgreSQL, deploy ke Railway 3. **Fullstack App** — Todo App dengan React + Express + JWT/OAuth, deploy penuh 4. **AI Agent App** — AI Chat Assistant dengan Mastra AI + Next.js 5. **Production Deploy** — Setup production-grade dengan Docker, VPS, CI/CD, monitoring

□ Peta Project

#	Project	Tech Stack	Deploy	Durasi
1	Landing Page Portfolio	HTML, CSS, Tailwind CSS	Vercel / Netlify	1 minggu

#	Project	Tech Stack	Deploy	Durasi
2	Bookshelf CRUD API	Express, TypeScript, Prisma, PostgreSQL	Railway	1 minggu
3	Todo App with Auth	React, Tailwind, Express, JWT, OAuth	Railway + Vercel	1-2 minggu
4	AI Chat Assistant	Mastra AI, OpenAI, Next.js	Vercel	1 minggu
5	Production Setup	Docker, VPS, Cloudflare, PM2, Sentry	VPS	1 minggu

□ Tujuan Pembelajaran

Setelah menyelesaikan modul ini, kamu akan mampu:

- **Mendeploy** aplikasi web ke berbagai platform (Vercel, Netlify, Railway, VPS)
- **Membangun REST API** production-ready dengan validasi, error handling, dan dokumentasi
- **Mengimplementasikan autentikasi** JWT dan OAuth Google
- **Membuat AI agent** dengan tool calling dan RAG memory
- **Mengkonfigurasi production deployment** dengan Docker, SSL, CI/CD, dan monitoring
- **Menulis README** dan dokumentasi API yang profesional
- **Menampilkan 5 proyek real** di portfolio GitHub

□ Prasyarat Teknis

- Node.js ≥ 20
- Git & GitHub
- PostgreSQL (bisa pakai Supabase atau Railway)
- Docker & Docker Compose
- Akun: GitHub, Vercel, Railway, Cloudflare
- OpenAI API key (untuk Project 4)
- VPS (bisa pakai DigitalOcean, Linode, atau Contabo sewa \$6/bulan)



Figure 1: Banner

□ Struktur Repositori

Setiap project bisa dibuat di repositori terpisah atau satu monorepo:

```
portfolio-project-series/  
├── 01-landing-page/      # Project 1  
├── 02-crud-api/          # Project 2  
├── 03-fullstack-app/     # Project 3  
├── 04-ai-agent-app/      # Project 4  
└── 05-production-deploy/ # Project 5
```

□ Cara Menggunakan Modul Ini

1. Baca README dan pahami deskripsi proyek
2. Ikuti step-by-step di setiap file sesi
3. Jangan copy-paste mentah — pahami setiap baris kode
4. Selesaikan latihan di akhir setiap sesi
5. Push ke GitHub setiap selesai satu milestone
6. Deploy dan dapatkan live URL

□ **Tip:** Satu repositori GitHub per project = 1 poin portfolio.
5 project = 5 poin. Lebih baik dari 1 repositori dengan 5 folder.

□ Kriteria Penilaian Portfolio

Kriteria	Bobot
Kode bersih & terstruktur	25%
Dokumentasi (README)	20%
Live demo berfungsi	25%
Responsive & UI/UX	15%
Fitur lengkap sesuai spesifikasi	15%

□ Referensi & Sumber Daya

- [Tailwind CSS Documentation](#)
- [Express.js Guide](#)
- [Prisma Documentation](#)
- [Mastra AI Docs](#)

- [Docker Documentation](#)
- [Railway Platform](#)
- [Vercel Deployment](#)

Selamat berkarya! ☑

Sesi 01: Landing Page Portfolio

Project 1 dari 5 — Landing page personal portfolio dengan HTML, CSS, dan Tailwind CSS

☑ Tujuan

- Membangun landing page personal portfolio dari nol
 - Menguasai Tailwind CSS utility-first framework
 - Membuat layout responsif (mobile-first)
 - Menyusun section: Hero, About, Skills, Projects, Contact
 - Mendeploy ke Vercel atau Netlify
-

☑ Deliverable

- Live URL: <https://namakamu.vercel.app> atau namakamu.netlify.app
 - GitHub repo: github.com/namakamu/portfolio
 - Domain custom (opsional)
-

☑ Tech Stack

Teknologi	Kegunaan
HTML5	Struktur halaman
Tailwind CSS (CDN)	Styling utility
Font Awesome / Heroicons	Ikon skills & sosial media
Formspree / Netlify Forms	Contact form backend
Vercel / Netlify	Hosting & deploy

□ Wireframe

NAVBAR (sticky)				
Logo	Home	About	Skills	Projects Contact
HERO SECTION				
Nama · Tagline · CTA Button				
ABOUT SECTION				
Foto	Paragraf singkat			
SKILLS SECTION				
[Ikon]	[Ikon]	[Ikon]	[Ikon]	
[Ikon]	[Ikon]	[Ikon]	[Ikon]	
PROJECTS GRID				
Card 1	Card 2	Card 3		
CONTACT FORM				
Nama	Email	Pesan	[Kirim]	
FOOTER				
Sosial Media · Copyright				

□ Langkah 1: Setup Proyek

Buat folder dan file index.html:

```
mkdir portfolio && cd portfolio
touch index.html
```

Gunakan Tailwind via CDN — tambahkan di <head>:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Nama Kamu – Portfolio</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <!-- Heroicons CDN -->
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@heroicons@1.0.2/dist/heroicons.min.css">
</head>
```

□ Langkah 2: Navbar

Navbar sticky dengan backdrop blur:

```
<nav class="fixed top-0 w-full bg-white/80 backdrop-blur-md z-50 shadow-sm">
  <div class="max-w-5xl mx-auto px-6 py-4 flex justify-between items-center">
    <a href="#" class="text-2xl font-bold text-gray-800">NamaKamu</a>
    <div class="hidden md:flex gap-8 text-gray-600 font-medium">
      <a href="#about" class="hover:text-blue-600 transition">About</a>
      <a href="#skills" class="hover:text-blue-600 transition">Skills</a>
      <a href="#projects" class="hover:text-blue-600 transition">Projects</a>
      <a href="#contact" class="hover:text-blue-600 transition">Contact</a>
    </div>
    <!-- Mobile hamburger -->
    <button id="menu-btn" class="md:hidden text-2xl">
      <i class="fas fa-bars"></i>
    </button>
  </div>
  <!-- Mobile menu -->
  <div id="mobile-menu" class="hidden md:hidden px-6 pb-4">
    <a href="#about" class="block py-2 text-gray-600">About</a>
    <a href="#skills" class="block py-2 text-gray-600">Skills</a>
    <a href="#projects" class="block py-2 text-gray-600">Projects</a>
    <a href="#contact" class="block py-2 text-gray-600">Contact</a>
  </div>
</nav>

<script>
  document.getElementById('menu-btn').addEventListener('click', () => {
    document.getElementById('mobile-menu').classList.toggle('hidden');
  });
</script>
```

□ Langkah 3: Hero Section

```
<section class="min-h-screen flex items-center justify-center bg-gradient-to-br
  <div class="text-center px-6">
    
      Halo, Saya <span class="text-blue-600">Nama Kamu</span>
    </h1>
    <p class="text-xl text-gray-600 mb-8 max-w-2xl mx-auto">
      Fullstack Developer & AI Enthusiast. Saya membangun aplikasi web mod
      dengan dampak nyata.
    </p>
    <div class="flex gap-4 justify-center">
      <a href="#projects"
        class="bg-blue-600 text-white px-8 py-3 rounded-full font-semibol
        Lihat Project
      </a>
      <a href="#contact"
        class="border-2 border-gray-300 text-gray-700 px-8 py-3 rounded-f
        Hubungi Saya
      </a>
    </div>
    <div class="flex gap-6 justify-center mt-8 text-2xl text-gray-500">
      <a href="#"><i class="fab fa-github hover:text-gray-900 transition">
      <a href="#"><i class="fab fa-linkedin hover:text-blue-700 transition"
      <a href="#"><i class="fab fa-twitter hover:text-blue-400 transition"
    </div>
  </div>
</section>
```

□ Langkah 4: About Section

```
<section id="about" class="py-20 px-6 max-w-5xl mx-auto">
  <h2 class="text-3xl font-bold text-center text-gray-900 mb-4">Tentang Saya</h2>
  <div class="w-20 h-1 bg-blue-600 mx-auto mb-12 rounded"></div>
  <div class="grid md:grid-cols-2 gap-12 items-center">
    <div>
      
    </div>
```



```

<div>
  <p class="text-gray-600 leading-relaxed mb-4">
    Saya adalah seorang developer dengan pengalaman dalam membangun
    dari frontend hingga backend. Saya percaya teknologi harus memberi
    bukan sekadar pamer fitur.
  </p>
  <p class="text-gray-600 leading-relaxed mb-6">
    Saat ini saya fokus pada fullstack JavaScript/TypeScript, cloud
    dan AI integration. Saya selalu ingin belajar hal baru dan berba
  </p>
  <a href="#" class="text-blue-600 font-semibold hover:underline">
    Download CV →
  </a>
</div>
</div>
</section>

```

□ Langkah 5: Skills Section

Gunakan grid ikon dengan label:

```

<section id="skills" class="py-20 bg-gray-50">
  <div class="max-w-5xl mx-auto px-6">
    <h2 class="text-3xl font-bold text-center text-gray-900 mb-4">Skills & T
    <div class="w-20 h-1 bg-blue-600 mx-auto mb-12 rounded"></div>
    <div class="grid grid-cols-2 md:grid-cols-4 gap-8">
      <div class="bg-white p-6 rounded-xl shadow-sm text-center hover:shad
        <i class="fab fa-html5 text-5xl text-orange-500 mb-3"></i>
        <p class="font-semibold text-gray-700">HTML</p>
      </div>
      <div class="bg-white p-6 rounded-xl shadow-sm text-center hover:shad
        <i class="fab fa-css3-alt text-5xl text-blue-500 mb-3"></i>
        <p class="font-semibold text-gray-700">CSS</p>
      </div>
      <div class="bg-white p-6 rounded-xl shadow-sm text-center hover:shad
        <i class="fab fa-js text-5xl text-yellow-500 mb-3"></i>
        <p class="font-semibold text-gray-700">JavaScript</p>
      </div>
      <div class="bg-white p-6 rounded-xl shadow-sm text-center hover:shad
        <i class="fab fa-react text-5xl text-cyan-500 mb-3"></i>
        <p class="font-semibold text-gray-700">React</p>
      </div>
      <div class="bg-white p-6 rounded-xl shadow-sm text-center hover:shad
        <i class="fab fa-node text-5xl text-green-600 mb-3"></i>

```

```

        <p class="font-semibold text-gray-700">Node.js</p>
    </div>
    <div class="bg-white p-6 rounded-xl shadow-sm text-center hover:shadow-sm">
        <i class="fab fa-python text-5xl text-blue-600 mb-3"></i>
        <p class="font-semibold text-gray-700">Python</p>
    </div>
    <div class="bg-white p-6 rounded-xl shadow-sm text-center hover:shadow-sm">
        <i class="fas fa-database text-5xl text-gray-700 mb-3"></i>
        <p class="font-semibold text-gray-700">PostgreSQL</p>
    </div>
    <div class="bg-white p-6 rounded-xl shadow-sm text-center hover:shadow-sm">
        <i class="fab fa-docker text-5xl text-blue-400 mb-3"></i>
        <p class="font-semibold text-gray-700">Docker</p>
    </div>
</div>
</div>
</section>

```

Langkah 6: Projects Grid

```

<section id="projects" class="py-20 px-6 max-w-5xl mx-auto">
    <h2 class="text-3xl font-bold text-center text-gray-900 mb-4">Project Terbaru
    <div class="w-20 h-1 bg-blue-600 mx-auto mb-12 rounded"></div>
    <div class="grid md:grid-cols-3 gap-8">
        <div class="bg-white rounded-xl shadow-md overflow-hidden hover:shadow-md">
            
            <div class="p-6">
                <h3 class="text-xl font-bold text-gray-900 mb-2">Project 1</h3>
                <p class="text-gray-600 mb-4">Deskripsi singkat project pertama.</p>
                <div class="flex gap-2 mb-4">
                    <span class="bg-blue-100 text-blue-700 text-xs px-2 py-1 rounded">Tag 1</span>
                    <span class="bg-green-100 text-green-700 text-xs px-2 py-1 rounded">Tag 2</span>
                </div>
                <a href="#" class="text-blue-600 font-semibold hover:underline">Detail Project 1</a>
            </div>
        </div>
        <!-- Ulangi untuk project 2 dan 3 -->
        <div class="bg-white rounded-xl shadow-md overflow-hidden hover:shadow-md">
            
            <div class="p-6">
                <h3 class="text-xl font-bold text-gray-900 mb-2">Project 2</h3>
                <p class="text-gray-600 mb-4">Deskripsi singkat project kedua.</p>
            </div>
        </div>
    </div>

```

```

        <div class="flex gap-2 mb-4">
          <span class="bg-purple-100 text-purple-700 text-xs px-2 py-1">
            <span class="bg-yellow-100 text-yellow-700 text-xs px-2 py-1">
              </div>
            <div>
              <a href="#" class="text-blue-600 font-semibold hover:underline">
                </div>
              </div>
            <div class="bg-white rounded-xl shadow-md overflow-hidden hover:shadow-x">
              
              <div class="p-6">
                <h3 class="text-xl font-bold text-gray-900 mb-2">Project 3</h3>
                <p class="text-gray-600 mb-4">Deskripsi singkat project ketiga.<
                  <div class="flex gap-2 mb-4">
                    <span class="bg-green-100 text-green-700 text-xs px-2 py-1 r">
                      <span class="bg-red-100 text-red-700 text-xs px-2 py-1 round">
                        </div>
                      <div>
                        <a href="#" class="text-blue-600 font-semibold hover:underline">
                          </div>
                        </div>
                      </div>
                    </div>
                  </section>

```

□ Langkah 7: Contact Form

Gunakan Formspree sebagai backend form:

```

<section id="contact" class="py-20 bg-gray-50">
  <div class="max-w-2xl mx-auto px-6">
    <h2 class="text-3xl font-bold text-center text-gray-900 mb-4">Hubungi Sa
    <div class="w-20 h-1 bg-blue-600 mx-auto mb-12 rounded"></div>
    <form action="https://formspree.io/f/yourFormID" method="POST"
      class="bg-white p-8 rounded-2xl shadow-md">
      <div class="mb-6">
        <label for="name" class="block text-gray-700 font-medium mb-2">N
        <input type="text" id="name" name="name" required
          class="w-full px-4 py-3 border border-gray-300 rounded-lg
        </div>
        <div class="mb-6">
          <label for="email" class="block text-gray-700 font-medium mb-2">
          <input type="email" id="email" name="_replyto" required
            class="w-full px-4 py-3 border border-gray-300 rounded-lg
          </div>
        <div class="mb-6">

```

```

        <label for="message" class="block text-gray-700 font-medium mb-2">
        <textarea id="message" name="message" rows="5" required
            class="w-full px-4 py-3 border border-gray-300 rounded
        </div>
        <button type="submit"
            class="w-full bg-blue-600 text-white py-3 rounded-lg font-se
            Kirim Pesan
        </button>
    </form>
</div>
</section>

```

Catatan: Daftar di [Formspree](#) dapatkan form ID, ganti yourFormID.

□ Langkah 8: Footer

```

<footer class="bg-gray-900 text-white py-8">
    <div class="max-w-5xl mx-auto px-6 text-center">
        <div class="flex gap-6 justify-center mb-4 text-2xl">
            <a href="#" class="hover:text-blue-400 transition"><i class="fab fa-
            <a href="#" class="hover:text-blue-400 transition"><i class="fab fa-
            <a href="#" class="hover:text-blue-400 transition"><i class="fab fa-
            <a href="#" class="hover:text-blue-400 transition"><i class="fab fa-
        </div>
        <p class="text-gray-400">© 2025 Nama Kamu. All rights reserved.</p>
    </div>
</footer>

```

□ Langkah 9: Deploy ke Vercel

1. Push ke GitHub:

```

git init
git add .
git commit -m "feat: portfolio landing page"
git remote add origin https://github.com/namakamu/portfolio.git
git push -u origin main

```

2. Buka [vercel.com](#) → Import GitHub repo
3. Vercel auto-detect static HTML → deploy selesai dalam 30 detik
4. Dapatkan URL: <https://portfolio-xxx.vercel.app>

Screenshot

Figure 2: Screenshot

❑ Langkah 10: Deploy ke Netlify (Alternatif)

1. Push ke GitHub
2. Buka netlify.com → Add new site → Import from GitHub
3. Deploy settings: Branch main, Publish directory /
4. Deploy

❑ Custom Domain (Opsional)

1. Beli domain di Niagahoster / Cloudflare
2. Di Vercel: Project → Settings → Domains → tambah domain
3. Di Netlify: Site settings → Domain management → Add custom domain
4. Setting DNS: A record atau CNAME sesuai petunjuk

❑ Screenshot Final

Portfolio Desktop

Figure 3: Portfolio Desktop

Portfolio Mobile

Figure 4: Portfolio Mobile

❑ Latihan

1. **Tambahkan animasi scroll** — gunakan [AOS Library](#)
2. **Dark mode toggle** — implementasikan dengan `class="dark"` dan Tailwind dark: prefix

3. **Testimonial section** — tambahkan section testimoni dengan layout carousel
 4. **Blog section** — hubungkan dengan dev.to API untuk menampilkan artikel terbaru
 5. **SEO meta tags** — tambahkan Open Graph dan Twitter Card meta tags
 6. **Performance** — ukur dengan Lighthouse, target score 90+
-

☐ Checklist

- ☐ Hero section dengan nama, tagline, CTA
 - ☐ About section dengan foto dan deskripsi
 - ☐ Skills section dengan ikon dan label
 - ☐ Projects grid minimal 3 card
 - ☐ Contact form dengan Formspree
 - ☐ Navbar sticky + mobile responsive
 - ☐ Footer dengan sosial media
 - ☐ Deploy ke Vercel/Netlify
 - ☐ GitHub repo dengan README
-

Project 1 selesai! Lanjut ke [Sesi 02: Bookshelf CRUD API](#)

Sesi 02: Bookshelf CRUD API

Project 2 dari 5 — REST API untuk Bookshelf (manajemen buku) dengan Express, TypeScript, Prisma, dan PostgreSQL

☐ Tujuan

- Membangun REST API dari nol dengan Express + TypeScript
 - Menggunakan Prisma ORM untuk database PostgreSQL
 - Implementasi validasi request dengan Zod
 - Menangani error secara terpusat (error handler middleware)
 - Mendeploy ke Railway
-

☐ Deliverable

- API Base URL: <https://bookshelf-api-production.up.railway.app>

- README dengan dokumentasi API lengkap
- GitHub repo: github.com/namakamu/bookshelf-api

□ Tech Stack

Teknologi	Kegunaan
Node.js + Express	HTTP server & routing
TypeScript	Type safety
Prisma ORM	Database management
PostgreSQL	Database relational
Zod	Request validation
Railway	Hosting & deploy

□ Endpoints

Method	Endpoint	Deskripsi
GET	/api/books	Ambil semua buku (dengan pagination & filter)
GET	/api/books/:id	Ambil detail buku
POST	/api/books	Tambah buku baru
PUT	/api/books/:id	Update data buku
DELETE	/api/books/:id	Hapus buku

□ Schema Database

```
Book
├── id: String (UUID)
├── title: String
├── author: String
├── isbn: String (unique)
├── publishedYear: Int
├── genre: String
├── description: String?
├── coverImage: String?
├── createdAt: DateTime
└── updatedAt: DateTime
```

□ Langkah 1: Setup Proyek

```
mkdir bookshelf-api && cd bookshelf-api
```

```
# Init project
```

```
npm init -y
```

```
# Install dependencies
```

```
npm install express prisma @prisma/client zod dotenv cors
```

```
# Install dev dependencies
```

```
npm install -D typescript ts-node-dev @types/express @types/cors @types/node tsx
```

```
# Init TypeScript
```

```
npx tsc --init
```

Update tsconfig.json:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "lib": ["ES2020"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist"]
}
```

□ Langkah 2: Setup Prisma

```
npx prisma init
```

Ini akan buat prisma/schema.prisma dan file .env.

Edit .env:


```
DATABASE_URL="postgresql://user:password@localhost:5432/bookshelf"
PORT=3000
```

Ganti user, password, dan localhost dengan koneksi PostgreSQL kamu. Untuk development, bisa pakai [Supabase](#) free tier.

Edit prisma/schema.prisma:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Book {
  id          String    @id @default(uuid())
  title       String
  author      String
  isbn        String    @unique
  publishedYear Int
  genre       String
  description  String?
  coverImage  String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

Jalankan migration:

```
npx prisma migrate dev --name init
```

□ Langkah 3: Struktur Folder

```
src/
├── index.ts           # Entry point
├── app.ts             # Express app setup
├── config/
│   └── env.ts         # Environment variables
├── lib/
│   └── prisma.ts      # Prisma client instance
├── routes/
│   └── book.routes.ts # Book routes
```

```

├── controllers/
│   └── book.controller.ts # Book handlers
├── services/
│   └── book.service.ts    # Business logic
├── schemas/
│   └── book.schema.ts     # Zod validation schemas
├── middlewares/
│   ├── error.middleware.ts # Global error handler
│   └── validate.middleware.ts # Validation middleware
└── types/
    └── index.ts           # Shared types

```

Buat semua folder:

```
mkdir -p src/{config,lib,routes,controllers,services,schemas,middlewares,types}
```

□ Langkah 4: Setup Express App

src/config/env.ts:

```

import dotenv from 'dotenv';
dotenv.config();

export const env = {
  port: parseInt(process.env.PORT || '3000', 10),
  databaseUrl: process.env.DATABASE_URL || '',
  nodeEnv: process.env.NODE_ENV || 'development',
};

```

src/lib/prisma.ts:

```

import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

export default prisma;

```

src/app.ts:

```

import express from 'express';
import cors from 'cors';
import bookRoutes from '../routes/book.routes';
import { errorMiddleware } from '../middlewares/error.middleware';

const app = express();

app.use(cors());

```

```

app.use(express.json());

// Routes
app.use('/api/books', bookRoutes);

// Health check
app.get('/health', (_req, res) => {
  res.json({ status: 'OK', timestamp: new Date().toISOString() });
});

// Error handler (harus setelah routes)
app.use(errorMiddleware);

export default app;

```

src/index.ts:

```

import app from './app';
import { env } from './config/env';

app.listen(env.port, () => {
  console.log(`☑ Server running on http://localhost:${env.port}`);
});

```

☐ Langkah 5: Zod Validation Schema

```

src/schemas/book.schema.ts:

import { z } from 'zod';

export const createBookSchema = z.object({
  body: z.object({
    title: z.string().min(1, 'Title is required').max(200),
    author: z.string().min(1, 'Author is required').max(100),
    isbn: z
      .string()
      .min(10, 'ISBN minimal 10 karakter')
      .max(13, 'ISBN maksimal 13 karakter'),
    publishedYear: z
      .number()
      .int()
      .min(1900, 'Tahun tidak valid')
      .max(new Date().getFullYear(), 'Tahun tidak boleh di masa depan'),
    genre: z.string().min(1, 'Genre is required'),
    description: z.string().max(2000).optional(),
  })
});

```

```

        coverImage: z.string().url('Cover image harus URL valid').optional(),
    })),
});

export const updateBookSchema = z.object({
  body: z.object({
    title: z.string().min(1).max(200).optional(),
    author: z.string().min(1).max(100).optional(),
    isbn: z.string().min(10).max(13).optional(),
    publishedYear: z
      .number()
      .int()
      .min(1900)
      .max(new Date().getFullYear())
      .optional(),
    genre: z.string().min(1).optional(),
    description: z.string().max(2000).optional(),
    coverImage: z.string().url().optional(),
  }),
});

export const paramsSchema = z.object({
  params: z.object({
    id: z.string().uuid('ID harus berupa UUID valid'),
  }),
});

export const querySchema = z.object({
  query: z.object({
    page: z.string().optional(),
    limit: z.string().optional(),
    genre: z.string().optional(),
    search: z.string().optional(),
  }),
});

```

□ Langkah 6: Validation Middleware

src/middlewares/validate.middleware.ts:

```

import { Request, Response, NextFunction } from 'express';
import { ZodSchema, ZodError } from 'zod';

export const validate =

```

```

(schema: ZodSchema) =>
(req: Request, res: Response, next: NextFunction) => {
  try {
    schema.parse({
      body: req.body,
      params: req.params,
      query: req.query,
    });
    next();
  } catch (error) {
    if (error instanceof ZodError) {
      return res.status(400).json({
        success: false,
        message: 'Validation error',
        errors: error.errors.map((e) => ({
          field: e.path.join('.'),
          message: e.message,
        })),
      });
    }
    next(error);
  }
};

```

□ Langkah 7: Service Layer

src/services/book.service.ts:

```

import prisma from '../lib/prisma';
import { Prisma } from '@prisma/client';

interface GetBooksParams {
  page: number;
  limit: number;
  genre?: string;
  search?: string;
}

export const bookService = {
  async getAll(params: GetBooksParams) {
    const { page, limit, genre, search } = params;
    const skip = (page - 1) * limit;

    const where: Prisma.BookWhereInput = {};

```

```

    if (genre) {
      where.genre = { equals: genre, mode: 'insensitive' };
    }

    if (search) {
      where.OR = [
        { title: { contains: search, mode: 'insensitive' } },
        { author: { contains: search, mode: 'insensitive' } },
        { isbn: { contains: search } },
      ];
    }

    const [books, total] = await Promise.all([
      prisma.book.findMany({
        where,
        skip,
        take: limit,
        orderBy: { createdAt: 'desc' },
      }),
      prisma.book.count({ where }),
    ]);

    return {
      success: true,
      data: books,
      meta: {
        page,
        limit,
        total,
        totalPages: Math.ceil(total / limit),
      },
    };
  },

  async getById(id: string) {
    const book = await prisma.book.findUnique({ where: { id } });
    if (!book) {
      throw new NotFoundError('Buku tidak ditemukan');
    }
    return { success: true, data: book };
  },

  async create(data: Prisma.BookCreateInput) {
    // Cek duplikat ISBN
    const existing = await prisma.book.findUnique({

```

```

        where: { isbn: data.isbn },
    });
    if (existing) {
        throw new ConflictError('ISBN sudah terdaftar');
    }

    const book = await prisma.book.create({ data });
    return { success: true, data: book };
},

async update(id: string, data: Prisma.BookUpdateInput) {
    const existing = await prisma.book.findUnique({ where: { id } });
    if (!existing) {
        throw new NotFoundError('Buku tidak ditemukan');
    }

    const book = await prisma.book.update({
        where: { id },
        data,
    });
    return { success: true, data: book };
},

async delete(id: string) {
    const existing = await prisma.book.findUnique({ where: { id } });
    if (!existing) {
        throw new NotFoundError('Buku tidak ditemukan');
    }

    await prisma.book.delete({ where: { id } });
    return { success: true, message: 'Buku berhasil dihapus' };
},
};

// Custom error classes
export class NotFoundError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'NotFoundError';
    }
}

export class ConflictError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'ConflictError';
    }
}

```

```
}  
}
```

□ Langkah 8: Controller

src/controllers/book.controller.ts:

```
import { Request, Response, NextFunction } from 'express';  
import { bookService } from '../services/book.service';  
  
export const bookController = {  
  async getAll(req: Request, res: Response, next: NextFunction) {  
    try {  
      const page = parseInt(req.query.page as string) || 1;  
      const limit = parseInt(req.query.limit as string) || 10;  
      const genre = req.query.genre as string | undefined;  
      const search = req.query.search as string | undefined;  
  
      const result = await bookService.getAll({ page, limit, genre, search });  
      res.json(result);  
    } catch (error) {  
      next(error);  
    }  
  },  
  
  async getById(req: Request, res: Response, next: NextFunction) {  
    try {  
      const result = await bookService.getById(req.params.id);  
      res.json(result);  
    } catch (error) {  
      next(error);  
    }  
  },  
  
  async create(req: Request, res: Response, next: NextFunction) {  
    try {  
      const result = await bookService.create(req.body);  
      res.status(201).json(result);  
    } catch (error) {  
      next(error);  
    }  
  },  
  
  async update(req: Request, res: Response, next: NextFunction) {
```



```

    try {
      const result = await bookService.update(req.params.id, req.body);
      res.json(result);
    } catch (error) {
      next(error);
    }
  },

  async delete(req: Request, res: Response, next: NextFunction) {
    try {
      const result = await bookService.delete(req.params.id);
      res.json(result);
    } catch (error) {
      next(error);
    }
  },
};

```

□ Langkah 9: Routes

src/routes/book.routes.ts:

```

import { Router } from 'express';
import { bookController } from '../controllers/book.controller';
import { validate } from '../middlewares/validate.middleware';
import {
  createBookSchema,
  updateBookSchema,
  paramsSchema,
  querySchema,
} from '../schemas/book.schema';

const router = Router();

router.get('/', validate(querySchema), bookController.getAll);
router.get('/:id', validate(paramsSchema), bookController.getById);
router.post('/', validate(createBookSchema), bookController.create);
router.put('/:id', validate(paramsSchema), validate(updateBookSchema), bookController.update);
router.delete('/:id', validate(paramsSchema), bookController.delete);

export default router;

```

□ Langkah 10: Global Error Handler

src/middlewares/error.middleware.ts:

```
import { Request, Response, NextFunction } from 'express';
import { NotFoundError, ConflictError } from '../services/book.service';

export const errorMiddleware = (
  err: Error,
  _req: Request,
  res: Response,
  _next: NextFunction
) => {
  console.error(`[ERROR] ${err.name}: ${err.message}`);

  if (err instanceof NotFoundError) {
    return res.status(404).json({
      success: false,
      message: err.message,
    });
  }

  if (err instanceof ConflictError) {
    return res.status(409).json({
      success: false,
      message: err.message,
    });
  }

  // Prisma known errors
  if (err.name === 'PrismaClientKnownRequestError') {
    return res.status(400).json({
      success: false,
      message: 'Database error',
    });
  }

  // Default 500
  res.status(500).json({
    success: false,
    message: 'Internal server error',
  });
};
```

□ Langkah 11: Scripts & Test

Update package.json:

```
{
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js",
    "db:migrate": "npx prisma migrate dev",
    "db:push": "npx prisma db push",
    "db:studio": "npx prisma studio",
    "seed": "tsx prisma/seed.ts"
  }
}
```

Jalankan development server:

```
npm run dev
```

Test dengan curl:

Create book

```
curl -X POST http://localhost:3000/api/books \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Atomic Habits",
    "author": "James Clear",
    "isbn": "9780735211292",
    "publishedYear": 2018,
    "genre": "Self-Improvement",
    "description": "An Easy & Proven Way to Build Good Habits & Break Bad Ones"
  }'
```

Get all books

```
curl http://localhost:3000/api/books
```

Get by ID (ganti dengan ID dari response create)

```
curl http://localhost:3000/api/books/<UUID>
```

Update

```
curl -X PUT http://localhost:3000/api/books/<UUID> \
  -H "Content-Type: application/json" \
  -d '{"title": "Atomic Habits - Updated"}'
```

Delete

```
curl -X DELETE http://localhost:3000/api/books/<UUID>
```

Screenshot

Figure 5: Screenshot

□ Langkah 12: Seed Data (Opsional)

prisma/seed.ts:

```
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

const books = [
  {
    title: 'Atomic Habits',
    author: 'James Clear',
    isbn: '9780735211292',
    publishedYear: 2018,
    genre: 'Self-Improvement',
    description: 'An Easy & Proven Way to Build Good Habits & Break Bad Ones',
  },
  {
    title: 'Clean Code',
    author: 'Robert C. Martin',
    isbn: '9780132350884',
    publishedYear: 2008,
    genre: 'Technology',
    description: 'A Handbook of Agile Software Craftsmanship',
  },
  {
    title: 'Sapiens',
    author: 'Yuval Noah Harari',
    isbn: '9780062316097',
    publishedYear: 2015,
    genre: 'History',
    description: 'A Brief History of Humankind',
  },
];

async function main() {
  console.log('Seeding...');
  for (const book of books) {
    await prisma.book.create({ data: book });
  }
}
```

```

    }
    console.log('Seeding done!');
  }

  main()
    .catch(console.error)
    .finally(() => prisma.$disconnect());

```

Jalankan:

```
npx tsx prisma/seed.ts
```

□ Langkah 13: Deploy ke Railway

1. Push ke GitHub:

```

git init
git add .
git commit -m "feat: bookshelf crud api"
git remote add origin https://github.com/namakamu/bookshelf-api.git
git push -u origin main

```

2. Buka railway.app → New Project → Deploy from GitHub repo

3. Tambahkan PostgreSQL:

- Railway: New → Database → PostgreSQL
- Railway auto-inject DATABASE_URL ke env

4. Tambahkan perintah:

- Railway dashboard → Settings
- Start Command: `npm run db:migrate && npm start`

5. Railway akan build + deploy otomatis

Screenshot

Figure 6: Screenshot

□ Dokumentasi API (README)

Buat README.md di root project:

Bookshelf API

REST API untuk manajemen buku. Dibangun dengan Express + TypeScript + Prisma.

Tech Stack

- Node.js + Express
- TypeScript
- Prisma ORM + PostgreSQL
- Zod Validation

API Documentation

****Base URL:**** ``https://bookshelf-api-production.up.railway.app``

Endpoints

Method	Endpoint	Deskripsi
GET	<code>`/api/books`</code>	List buku (pagination + filter)
GET	<code>`/api/books/:id`</code>	Detail buku
POST	<code>`/api/books`</code>	Tambah buku
PUT	<code>`/api/books/:id`</code>	Update buku
DELETE	<code>`/api/books/:id`</code>	Hapus buku

Query Parameters (GET /api/books)

Param	Type	Default	Deskripsi
page	number	1	Halaman
limit	number	10	Item per halaman
genre	string	-	Filter genre
search	string	-	Cari judul/penulis

Contoh Response

```
``json
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "title": "Atomic Habits",
      "author": "James Clear",
      ...
    }
  ]
}
```

```

    ],
    "meta": {
      "page": 1,
      "limit": 10,
      "total": 1,
      "totalPages": 1
    }
  }
}

```

□ Latihan

1. **Filter genres** – buat endpoint `GET /api/genres` yang return daftar genre unik
2. **Sorting** – tambahkan query param `sortBy` dan `sortOrder`
3. **Review model** – tambahkan model `Review` relasi ke `Book`, buat CRUD review
4. **Rate limiting** – implementasikan `express-rate-limit` untuk mencegah spam
5. **Swagger docs** – setup `swagger-jsdoc` + `swagger-ui-express` untuk dokumentasi API interaktif
6. **Unit test** – tulis test dengan Jest + Supertest untuk setiap endpoint

□ Checklist

- [] Project TypeScript setup dengan Express
- [] Prisma schema + migration berhasil
- [] Semua 5 endpoint berfungsi
- [] Validation dengan Zod berjalan
- [] Error handler menangani NotFound, Conflict, Validation
- [] Pagination & filter bekerja
- [] Deploy ke Railway
- [] README dengan dokumentasi API lengkap

> **Project 2 selesai!** Lanjut ke [Sesi 03: Todo App with Auth](./03-fullstack-app.md)

Sesi 03: Todo App with Auth

> **Project 3 dari 5** – Aplikasi Todo fullstack dengan autentikasi JWT + OAuth Google

□ Tujuan

- Membangun fullstack app dari nol (backend + frontend)
- Implementasi autentikasi JWT (register, login, protected routes)
- Integrasi OAuth Google Login
- CRUD todos dengan relasi ke user
- Filter, search, dan pagination
- Deploy backend ke Railway + frontend ke Vercel

□ Deliverable

- Live fullstack app: ``https://todo-app-namakamu.vercel.app``
- GitHub repo: ``github.com/namakamu/todo-app``
- Backend: Railway, Frontend: Vercel

□ Tech Stack

Backend

Teknologi	Kegunaan
Express + TypeScript	REST API
Prisma + PostgreSQL	Database
JWT (jsonwebtoken)	Token-based auth
Passport.js + Google Strategy	OAuth Google
Zod	Validation
bcryptjs	Password hashing

Frontend

Teknologi	Kegunaan
React + Vite	UI Framework
Tailwind CSS	Styling
React Router	Routing
React Query / TanStack Query	Data fetching
Axios	HTTP client
Zustand / Context API	State management

Database Schema

User |— id: String (UUID) |— email: String (unique) |— password: String? (nullable untuk OAuth) |— name: String |— avatar: String? |— googleId: String? (unique) |— createdAt: DateTime |— updatedAt: DateTime

Todo |— id: String (UUID) |— title: String |— description: String? |— status: enum (PENDING, IN_PROGRESS, COMPLETED, CANCELLED) |— priority: enum (LOW, MEDIUM, HIGH) |— category: String? |— dueDate: DateTime? |— userId: String (FK → User) |— createdAt: DateTime |— updatedAt: DateTime

Bagian 1: Setup Backend

Langkah 1.1: Init Project

```
```bash
mkdir todo-api && cd todo-api
npm init -y
npm install express prisma @prisma/client zod dotenv cors jsonwebtoken bcryptjs passport-google-oauth20
npm install -D typescript @types/express @types/cors @types/jsonwebtoken @types/bcryptjs @types/passport-google-oauth20 tsx
npx tsc --init
mkdir -p src/{config,lib,routes,controllers,services,schemas,middlewares,types}
npx prisma init
```

### Langkah 1.2: Prisma Schema

```
prisma/schema.prisma:

generator client {
 provider = "prisma-client-js"
}

datasource db {
 provider = "postgresql"
 url = env("DATABASE_URL")
}

enum TodoStatus {
 PENDING
 IN_PROGRESS
```

```

 COMPLETED
 CANCELLED
 }

enum TodoPriority {
 LOW
 MEDIUM
 HIGH
}

model User {
 id String @id @default(uuid())
 email String @unique
 password String?
 name String
 avatar String?
 googleId String? @unique
 todos Todo[]
 createdAt DateTime @default(now())
 updatedAt DateTime @updatedAt
}

model Todo {
 id String @id @default(uuid())
 title String
 description String?
 status TodoStatus @default(PENDING)
 priority TodoPriority @default(MEDIUM)
 category String?
 dueDate DateTime?
 userId String
 user User @relation(fields: [userId], references: [id], onDelete=
 createdAt DateTime @default(now())
 updatedAt DateTime @updatedAt
}

```

Jalankan migration:

```
npx prisma migrate dev --name init
```

### Langkah 1.3: Environment Variables

.env:

```

DATABASE_URL="postgresql://user:***@localhost:5432/todoapp"
JWT_SECRET="your-super-secret-key-change-in-production"
JWT_EXPIRES_IN="7d"

```

```
PORT=3001
FRONTEND_URL="http://localhost:5173"
```

```
Google OAuth
GOOGLE_CLIENT_ID="your-google-client-id"
GOOGLE_CLIENT_SECRET="your-google-client-secret"
GOOGLE_CALLBACK_URL="http://localhost:3001/api/auth/google/callback"
```

**Google OAuth Setup:** Buka [Google Cloud Console](#) → Credentials → OAuth 2.0 Client IDs → Buat Web application dengan authorized redirect URI <http://localhost:3001/api/auth/google/callback>

#### Langkah 1.4: Prisma Client & Env Config

src/lib/prisma.ts:

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();
export default prisma;
```

src/config/env.ts:

```
import dotenv from 'dotenv';
dotenv.config();

export const env = {
 port: parseInt(process.env.PORT || '3001', 10),
 databaseUrl: process.env.DATABASE_URL || '',
 jwtSecret: process.env.JWT_SECRET || 'fallback-secret',
 jwtExpiresIn: process.env.JWT_EXPIRES_IN || '7d',
 frontendUrl: process.env.FRONTEND_URL || 'http://localhost:5173',
 googleClientId: process.env.GOOGLE_CLIENT_ID || '',
 googleClientSecret: process.env.GOOGLE_CLIENT_SECRET || '',
 googleCallbackUrl: process.env.GOOGLE_CALLBACK_URL || '',
};
```

#### Langkah 1.5: Auth Service

src/services/auth.service.ts:

```
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import prisma from '../lib/prisma';
import { env } from '../config/env';

export const authService = {
 async register(email: string, password: string, name: string) {
 const existing = await prisma.user.findUnique({ where: { email } });
```

```

 if (existing) {
 throw new Error('Email sudah terdaftar');
 }

 const hashedPassword = await bcrypt.hash(password, 12);
 const user = await prisma.user.create({
 data: { email, password: hashedPassword, name },
 });

 const token = this.generateToken(user.id);
 return { token, user: { id: user.id, name: user.name, email: user.email } };
 },

 async login(email: string, password: string) {
 const user = await prisma.user.findUnique({ where: { email } });
 if (!user || !user.password) {
 throw new Error('Email atau password salah');
 }

 const valid = await bcrypt.compare(password, user.password);
 if (!valid) {
 throw new Error('Email atau password salah');
 }

 const token = this.generateToken(user.id);
 return { token, user: { id: user.id, name: user.name, email: user.email, avatar: user.avatar } },

 async loginWithGoogle(profile: { id: string; email: string; name: string; avatar: string }) {
 let user = await prisma.user.findFirst({
 where: { OR: [{ googleId: profile.id }, { email: profile.email }] },
 });

 if (user) {
 // Link Google ID jika belum ter-link
 if (!user.googleId) {
 user = await prisma.user.update({
 where: { id: user.id },
 data: { googleId: profile.id, avatar: profile.avatar || user.avatar },
 });
 }
 } else {
 // Buat user baru
 user = await prisma.user.create({
 data: {
 email: profile.email,

```

```

 name: profile.name,
 googleId: profile.id,
 avatar: profile.avatar,
 },
 });
 }

 const token = this.generateToken(user.id);
 return { token, user: { id: user.id, name: user.name, email: user.email, avatar: user.avatar } },
},

async getProfile(userId: string) {
 const user = await prisma.user.findUnique({
 where: { id: userId },
 select: { id: true, name: true, email: true, avatar: true, createdAt: true },
 });
 if (!user) throw new Error('User tidak ditemukan');
 return user;
},

generateToken(userId: string) {
 return jwt.sign({ userId }, env.jwtSecret, { expiresIn: env.jwtExpiresIn });
},
};

```

### Langkah 1.6: Auth Middleware

src/middlewares/auth.middleware.ts:

```

import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import { env } from '../config/env';

export interface AuthRequest extends Request {
 userId?: string;
}

export const authMiddleware = (
 req: AuthRequest,
 res: Response,
 next: NextFunction
) => {
 const header = req.headers.authorization;
 if (!header || !header.startsWith('Bearer ')) {
 return res.status(401).json({ success: false, message: 'Token tidak ditemukan' });
 }
 next();
};

```

```

const token = header.split(' ')[1];

try {
 const decoded = jwt.verify(token, env.jwtSecret) as { userId: string };
 req.userId = decoded.userId;
 next();
} catch {
 return res.status(401).json({ success: false, message: 'Token tidak valid' });
}
};

```

### Langkah 1.7: Auth Routes

src/routes/auth.routes.ts:

```

import { Router } from 'express';
import { authController } from '../controllers/auth.controller';
import { authMiddleware } from '../middlewares/auth.middleware';

const router = Router();

router.post('/register', authController.register);
router.post('/login', authController.login);
router.get('/google', authController.googleAuth);
router.get('/google/callback', authController.googleCallback);
router.get('/me', authMiddleware, authController.getProfile);

export default router;

```

### Langkah 1.8: Auth Controller

src/controllers/auth.controller.ts:

```

import { Request, Response, NextFunction } from 'express';
import passport from 'passport';
import { authService } from '../services/auth.service';
import { AuthRequest } from '../middlewares/auth.middleware';

export const authController = {
 async register(req: Request, res: Response, next: NextFunction) {
 try {
 const { email, password, name } = req.body;
 const result = await authService.register(email, password, name);
 res.status(201).json({ success: true, ...result });
 } catch (error: any) {

```

```

 res.status(400).json({ success: false, message: error.message });
 }
},

async login(req: Request, res: Response, next: NextFunction) {
 try {
 const { email, password } = req.body;
 const result = await authService.login(email, password);
 res.json({ success: true, ...result });
 } catch (error: any) {
 res.status(401).json({ success: false, message: error.message });
 }
},

googleAuth: passport.authenticate('google', {
 scope: ['profile', 'email'],
 session: false,
}),

googleCallback: (req: Request, res: Response, next: NextFunction) => {
 passport.authenticate('google', { session: false }, async (err: any, profile: Profile) => {
 if (err || !profile) {
 return res.redirect(`${process.env.FRONTEND_URL}/login?error=auth_failed`);
 }
 try {
 const result = await authService.loginWithGoogle({
 id: profile.id,
 email: profile.emails?.[0]?.value || '',
 name: profile.displayName,
 avatar: profile.photos?.[0]?.value,
 });
 // Redirect ke frontend dengan token
 res.redirect(`${process.env.FRONTEND_URL}/auth/callback?token=${result.token}`);
 } catch (error) {
 res.redirect(`${process.env.FRONTEND_URL}/login?error=auth_failed`);
 }
 })(req, res, next);
},

async getProfile(req: AuthRequest, res: Response, next: NextFunction) {
 try {
 const user = await authService.getProfile(req.userId!);
 res.json({ success: true, data: user });
 } catch (error: any) {
 res.status(404).json({ success: false, message: error.message });
 }
}

```

```
 },
 };
};
```

### Langkah 1.9: Passport Google Strategy

src/config/passport.ts:

```
import passport from 'passport';
import { Strategy as GoogleStrategy } from 'passport-google-oauth20';
import { env } from './env';

passport.use(
 new GoogleStrategy(
 {
 clientID: env.googleClientId,
 clientSecret: env.googleClientSecret,
 callbackURL: env.googleCallbackUrl,
 },
 (_accessToken, _refreshToken, profile, done) => {
 done(null, profile);
 }
)
);

export default passport;
```

### Langkah 1.10: Todo Service

src/services/todo.service.ts:

```
import prisma from '../lib/prisma';
import { Prisma, TodoStatus, TodoPriority } from '@prisma/client';

interface GetTodosParams {
 userId: string;
 page: number;
 limit: number;
 status?: TodoStatus;
 priority?: TodoPriority;
 category?: string;
 search?: string;
}

export const todoService = {
 async getAll(params: GetTodosParams) {
 const { userId, page, limit, status, priority, category, search } = params;
```



```

const skip = (page - 1) * limit;

const where: Prisma.TODOWhereInput = { userId };

if (status) where.status = status;
if (priority) where.priority = priority;
if (category) where.category = category;
if (search) {
 where.OR = [
 { title: { contains: search, mode: 'insensitive' } },
 { description: { contains: search, mode: 'insensitive' } },
];
}

const [todos, total] = await Promise.all([
 prisma.todo.findMany({
 where,
 skip,
 take: limit,
 orderBy: { createdAt: 'desc' },
 }),
 prisma.todo.count({ where }),
]);

return {
 success: true,
 data: todos,
 meta: { page, limit, total, totalPages: Math.ceil(total / limit) },
};
},

async getById(id: string, userId: string) {
 const todo = await prisma.todo.findFirst({
 where: { id, userId },
 });
 if (!todo) throw new Error('Todo tidak ditemukan');
 return { success: true, data: todo };
},

async create(data: { title: string; description?: string; priority?: TodoPriority }) {
 const todo = await prisma.todo.create({
 data: {
 ...data,
 dueDate: data.dueDate ? new Date(data.dueDate) : undefined,
 userId,
 },
 },

```

```

 });
 return { success: true, data: todo };
 },

 async update(id: string, userId: string, data: Prisma.TODOUpdateInput) {
 const existing = await prisma.todo.findFirst({ where: { id, userId } });
 if (!existing) throw new Error('Todo tidak ditemukan');

 const todo = await prisma.todo.update({ where: { id }, data });
 return { success: true, data: todo };
 },

 async delete(id: string, userId: string) {
 const existing = await prisma.todo.findFirst({ where: { id, userId } });
 if (!existing) throw new Error('Todo tidak ditemukan');

 await prisma.todo.delete({ where: { id } });
 return { success: true, message: 'Todo berhasil dihapus' };
 },

 async getCategories(userId: string) {
 const result = await prisma.todo.groupBy({
 by: ['category'],
 where: { userId, category: { not: null } },
 });
 return { success: true, data: result.map((r) => r.category) };
 },
};

```

### Langkah 1.11: Todo Routes & Controller

src/routes/todo.routes.ts:

```

import { Router } from 'express';
import { todoController } from '../controllers/todo.controller';
import { authMiddleware } from '../middlewares/auth.middleware';

const router = Router();

router.use(authMiddleware); // Semua todo routes butuh auth

router.get('/', todoController.getAll);
router.get('/categories', todoController.getCategories);
router.get('/:id', todoController.getById);
router.post('/', todoController.create);
router.put('/:id', todoController.update);

```

```

router.delete('/:id', todoController.delete);

export default router;
src/controllers/todo.controller.ts:
import { Response, NextFunction } from 'express';
import { todoService } from '../services/todo.service';
import { AuthRequest } from '../middlewares/auth.middleware';

export const todoController = {
 async getAll(req: AuthRequest, res: Response, next: NextFunction) {
 try {
 const page = parseInt(req.query.page as string) || 1;
 const limit = parseInt(req.query.limit as string) || 10;
 const result = await todoService.getAll({
 userId: req.userId!,
 page,
 limit,
 status: req.query.status as any,
 priority: req.query.priority as any,
 category: req.query.category as string,
 search: req.query.search as string,
 });
 res.json(result);
 } catch (error) { next(error); }
 },

 async getById(req: AuthRequest, res: Response, next: NextFunction) {
 try {
 const result = await todoService.getById(req.params.id, req.userId!);
 res.json(result);
 } catch (error) { next(error); }
 },

 async create(req: AuthRequest, res: Response, next: NextFunction) {
 try {
 const result = await todoService.create(req.body, req.userId!);
 res.status(201).json(result);
 } catch (error) { next(error); }
 },

 async update(req: AuthRequest, res: Response, next: NextFunction) {
 try {
 const result = await todoService.update(req.params.id, req.userId!, req.bo
 res.json(result);
 } catch (error) { next(error); }
 }
};

```

```

 },

 async delete(req: AuthRequest, res: Response, next: NextFunction) {
 try {
 const result = await todoService.delete(req.params.id, req.userId!);
 res.json(result);
 } catch (error) { next(error); }
 },

 async getCategories(req: AuthRequest, res: Response, next: NextFunction) {
 try {
 const result = await todoService.getCategories(req.userId!);
 res.json(result);
 } catch (error) { next(error); }
 },
 },
};

```

### Langkah 1.12: Main App

src/app.ts:

```

import express from 'express';
import cors from 'cors';
import passport from 'passport';
import './config/passport';
import authRoutes from './routes/auth.routes';
import todoRoutes from './routes/todo.routes';

const app = express();

app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));
app.use(express.json());
app.use(passport.initialize());

app.use('/api/auth', authRoutes);
app.use('/api/todos', todoRoutes);

app.get('/health', (_req, res) => res.json({ status: 'OK' }));

export default app;

```

---

## ▣ Bagian 2: Setup Frontend

### Langkah 2.1: Init React + Vite

```
npm create vite@latest todo-app -- --template react-ts
cd todo-app
npm install
npm install tailwindcss @tailwindcss/vite axios react-router-dom @tanstack/react
```

### Langkah 2.2: Tailwind Config

vite.config.ts:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
```

```
export default defineConfig({
 plugins: [react(), tailwindcss()],
})
```

src/index.css:

```
@import "tailwindcss";
```

### Langkah 2.3: API Client

src/lib/api.ts:

```
import axios from 'axios';
```

```
const api = axios.create({
 baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3001',
});
```

*// Interceptor untuk attach token*

```
api.interceptors.request.use((config) => {
 const token = localStorage.getItem('token');
 if (token) {
 config.headers.Authorization = `Bearer ${token}`;
 }
 return config;
});
```

*// Interceptor untuk handle 401*

```
api.interceptors.response.use(
 (response) => response,
 (error) => {
```

```

 if (error.response?.status === 401) {
 localStorage.removeItem('token');
 window.location.href = '/login';
 }
 return Promise.reject(error);
 }
);

```

```
export default api;
```

## Langkah 2.4: Auth Context

src/context/AuthContext.tsx:

```
import { createContext, useContext, useState, useEffect, ReactNode } from 'react'
import api from '../lib/api';
```

```
interface User {
 id: string;
 name: string;
 email: string;
 avatar?: string;
}
```

```
interface AuthContextType {
 user: User | null;
 token: string | null;
 login: (email: string, password: string) => Promise<void>;
 register: (email: string, password: string, name: string) => Promise<void>;
 logout: () => void;
 loading: boolean;
}
```

```
const AuthContext = createContext<AuthContextType | null>(null);
```

```
export function AuthProvider({ children }: { children: ReactNode }) {
 const [user, setUser] = useState<User | null>(null);
 const [token, setToken] = useState<string | null>(localStorage.getItem('token') || null);
 const [loading, setLoading] = useState(true);
```

```
 useEffect(() => {
 if (token) {
 api.get('/api/auth/me')
 .then((res) => setUser(res.data.data))
 .catch(() => {
 localStorage.removeItem('token');
 });
 }
 }, [token]);
}
```

```

 setToken(null);
 })
 .finally(() => setLoading(false));
 } else {
 setLoading(false);
 }
 }, [token]));

const login = async (email: string, password: string) => {
 const res = await api.post('/api/auth/login', { email, password });
 localStorage.setItem('token', res.data.token);
 setToken(res.data.token);
 setUser(res.data.user);
};

const register = async (email: string, password: string, name: string) => {
 const res = await api.post('/api/auth/register', { email, password, name });
 localStorage.setItem('token', res.data.token);
 setToken(res.data.token);
 setUser(res.data.user);
};

const logout = () => {
 localStorage.removeItem('token');
 setToken(null);
 setUser(null);
};

const googleLogin = () => {
 window.location.href = `${import.meta.env.VITE_API_URL}/api/auth/google`;
};

return (
 <AuthContext.Provider value={{ user, token, login, register, logout, loading }}>
 {children}
 </AuthContext.Provider>
);
}

export const useAuth = () => {
 const ctx = useContext(AuthContext);
 if (!ctx) throw new Error('useAuth must be used within AuthProvider');
 return ctx;
};

```

## Langkah 2.5: Login Page

src/pages/Login.tsx:

```
import { useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { Link, useNavigate } from 'react-router-dom';

export default function Login() {
 const [email, setEmail] = useState('');
 const [password, setPassword] = useState('');
 const [error, setError] = useState('');
 const { login } = useAuth();
 const navigate = useNavigate();

 const handleSubmit = async (e: React.FormEvent) => {
 e.preventDefault();
 try {
 await login(email, password);
 navigate('/');
 } catch (err: any) {
 setError(err.response?.data?.message || 'Login gagal');
 }
 };

 return (
 <div className="min-h-screen flex items-center justify-center bg-gray-50">
 <div className="bg-white p-8 rounded-2xl shadow-md w-full max-w-md">
 <h2 className="text-2xl font-bold mb-6 text-center">Masuk</h2>
 {error && <div className="bg-red-100 text-red-700 p-3 rounded-lg mb-4">{error}</div>}
 <form onSubmit={handleSubmit} className="space-y-4">
 <div>
 <label className="block text-sm font-medium mb-1">Email</label>
 <input type="email" value={email} onChange={(e) => setEmail(e.target.value)}
 className="w-full px-4 py-2 border rounded-lg focus:ring-2 focus:ring-blue-500"/>
 </div>
 <div>
 <label className="block text-sm font-medium mb-1">Password</label>
 <input type="password" value={password} onChange={(e) => setPassword(e.target.value)}
 className="w-full px-4 py-2 border rounded-lg focus:ring-2 focus:ring-blue-500"/>
 </div>
 <button type="submit" className="w-full bg-blue-600 text-white py-2 rounded-lg">
 Masuk
 </button>
 </form>
 <div className="mt-4 text-center">

```



```

 <button className="w-full border py-2 rounded-lg hover:bg-gray-50 tran

 </div>
 <p className="text-center mt-4 text-sm text-gray-600">
 Belum punya akun? <Link to="/register" className="text-blue-600 hover:
 </p>
 </div>
 </div>
);
}

```

## Langkah 2.6: Todo Dashboard

src/pages/Dashboard.tsx:

```

import { useState } from 'react';
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import api from '../lib/api';
import { useAuth } from '../context/AuthContext';

export default function Dashboard() {
 const { user, logout } = useAuth();
 const queryClient = useQueryClient();
 const [title, setTitle] = useState('');
 const [statusFilter, setStatusFilter] = useState('');

 const { data, isLoading } = useQuery({
 queryKey: ['todos', statusFilter],
 queryFn: () => api.get(`/api/todos${statusFilter ? `?status=${statusFilter}`}`),
 });

 const createMutation = useMutation({
 mutationFn: (title: string) => api.post('/api/todos', { title }),
 onSuccess: () => {
 queryClient.invalidateQueries({ queryKey: ['todos'] });
 setTitle('');
 },
 });

 const deleteMutation = useMutation({
 mutationFn: (id: string) => api.delete(`/api/todos/${id}`),
 onSuccess: () => queryClient.invalidateQueries({ queryKey: ['todos'] }),
 });
}

```

```
const updateMutation = useMutation({
 mutationFn: ({ id, data }: { id: string; data: any }) => api.put(`/api/todos/
 onSuccess: () => queryClient.invalidateQueries({ queryKey: ['todos'] })),
});

const handleSubmit = (e: React.FormEvent) => {
 e.preventDefault();
 if (title.trim()) createMutation.mutate(title);
};

return (
 <div className="min-h-screen bg-gray-50">
 <nav className="bg-white shadow-sm p-4 flex justify-between items-center m
 <h1 className="text-xl font-bold">Todo App</h1>
 <div className="flex items-center gap-4">
 {user?.name}
 <button onClick={logout} className="text-red-600 hover:underline">Kelu
 </div>
 </nav>

 <main className="max-w-4xl mx-auto p-4">
 {/* Add Todo Form */}
 <form onSubmit={handleSubmit} className="flex gap-2 mb-6">
 <input
 type="text"
 value={title}
 onChange={(e) => setTitle(e.target.value)}
 placeholder="Tambah todo baru..."
 className="flex-1 px-4 py-2 border rounded-lg focus:ring-2 focus:r
 />
 <button type="submit"
 className="bg-blue-600 text-white px-6 py-2 rounded-lg hover:bg-bl
 Tambah
 </button>
 </form>

 {/* Status Filter */}
 <div className="flex gap-2 mb-4">
 [{ '', 'PENDING', 'IN_PROGRESS', 'COMPLETED' }.map((s) => (
 <button
 key={s}
 onClick={() => setStatusFilter(s)}
 className={`px-3 py-1 rounded-full text-sm ${
 statusFilter === s ? 'bg-blue-600 text-white' : 'bg-gray-200 tex
```

```
const handleSubmit = (e: React.FormEvent) => {
 e.preventDefault();
 if (title.trim()) createMutation.mutate(title);
};
```

```
return (
 <div className="min-h-screen bg-gray-50">
 <nav className="bg-white shadow-sm p-4 flex justify-between items-center m">
 <h1 className="text-xl font-bold">Todo App</h1>
 <div className="flex items-center gap-4">
 {user?.name}
 <button onClick={logout} className="text-red-600 hover:underline">Keluar</button>
 </div>
 </nav>
 </div>
);
```

```
<main className="max-w-4xl mx-auto p-4">
 {/* Add Todo Form */}
 <form onSubmit={handleSubmit} className="flex gap-2 mb-6">
 <input
 type="text"
 value={title}
 onChange={(e) => setTitle(e.target.value)}
 placeholder="Tambah todo baru..."
 className="flex-1 px-4 py-2 border rounded-lg focus:ring-2 focus:ring-blue-500"
 />
 <button type="submit"
 className="bg-blue-600 text-white px-6 py-2 rounded-lg hover:bg-blue-700"
 >
 Tambah
 </button>
 </form>

```

```

{ /* Status Filter */}
<div className="flex gap-2 mb-4">
 {['', 'PENDING', 'IN_PROGRESS', 'COMPLETED'].map((s) => (
 <button
 key={s}
 onClick={() => setStatusFilter(s)}
 className={`px-3 py-1 rounded-full text-sm ${
 statusFilter === s ? 'bg-blue-600 text-white' : 'bg-gray-200 tex
 }`}
 >

```

```

 {s || 'Semua'}
 </button>
)}}
 </div>

 {/* Todo List */}
 {isLoading ? (
 <p className="text-center text-gray-500">Memuat...</p>
) : (
 <div className="space-y-3">
 {data?.data?.data?.map((todo: any) => (
 <div key={todo.id} className="bg-white p-4 rounded-lg shadow-sm fl
 <div className="flex items-center gap-3">
 <input
 type="checkbox"
 checked={todo.status === 'COMPLETED'}
 onChange={() => updateMutation.mutate({
 id: todo.id,
 data: { status: todo.status === 'COMPLETED' ? 'PENDING' :
 })}
 className="w-5 h-5"
 />
 <div>
 <p className={`font-medium ${todo.status === 'COMPLETED' ? '
 {todo.title}
 </p>
 <p className="text-xs text-gray-400">
 {todo.priority} · {todo.category || 'Uncategorized'}
 </p>
 </div>
 </div>
 <button
 onClick={() => deleteMutation.mutate(todo.id)}
 className="text-red-500 hover:text-red-700">
 Hapus
 </button>
 </div>
)}}
 {data?.data?.data?.length === 0 && (
 <p className="text-center text-gray-400">Belum ada todo. Tambahkan
)}
 </div>
)}
</main>
</div>
);

```

```
}
```

## Langkah 2.7: App Router

src/App.tsx:

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { AuthProvider, useAuth } from './context/AuthContext';
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';

const queryClient = new QueryClient();

function ProtectedRoute({ children }: { children: React.ReactNode }) {
 const { token, loading } = useAuth();
 if (loading) return <div className="min-h-screen flex items-center justify-center"><div>Loading...</div></div>;
 return token ? children : <Navigate to="/login" />;
}

function App() {
 return (
 <QueryClientProvider client={queryClient}>
 <BrowserRouter>
 <AuthProvider>
 <Routes>
 <Route path="/login" element={<Login />} />
 <Route path="/register" element={<Register />} />
 <Route path="/" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
 </Routes>
 </AuthProvider>
 </BrowserRouter>
 </QueryClientProvider>
);
}

export default App;
```

---

## □ Langkah 3: Deploy

### Backend → Railway

1. Push backend ke GitHub

2. Railway: New Project → Deploy from GitHub
3. Tambah PostgreSQL
4. Set environment variables di Railway:
  - JWT\_SECRET, FRONTEND\_URL (URL Vercel nanti)
  - GOOGLE\_CLIENT\_ID, GOOGLE\_CLIENT\_SECRET
  - Update GOOGLE\_CALLBACK\_URL ke `https://todo-api-production.up.railway.app/api/auth/google/callback`
5. Start command: `npx prisma migrate deploy && npm start`

### Frontend → Vercel

1. Buat `.env.production`:  
`VITE_API_URL=https://todo-api-production.up.railway.app`
2. Push frontend ke GitHub
3. Vercel: Import repo → Deploy
4. Set VITE\_API\_URL di Vercel Environment Variables

Screenshot

Figure 7: Screenshot

---

### □ Latihan

1. **Due date picker** — tambahkan input date untuk due date, tampilkan todo yang overdue
2. **Drag & drop** — implementasikan drag and drop untuk mengubah status todo
3. **Filter by category** — dropdown filter kategori dinamis
4. **Export data** — tambahkan tombol export todos ke CSV
5. **Dark mode** — implementasikan dark mode toggle
6. **Pagination UI** — tambahkan navigasi halaman di frontend
7. **Toast notification** — tampilkan notifikasi sukses/gagal
8. **PWA** — buat todo app jadi Progressive Web App

---

### □ Checklist

- ☐ Backend: Register & Login dengan JWT
- ☐ Backend: OAuth Google login
- ☐ Backend: CRUD todos dengan autentikasi

- ☐ Backend: Filter & pagination
- ☐ Frontend: Login & Register page
- ☐ Frontend: Dashboard dengan daftar todo
- ☐ Frontend: Form tambah todo
- ☐ Frontend: Update status (checklist)
- ☐ Frontend: Hapus todo
- ☐ Frontend: Protected routes
- ☐ Deploy backend ke Railway
- ☐ Deploy frontend ke Vercel
- ☐ Auth flow: register → login → CRUD → logout

---

**Project 3 selesai!** Lanjut ke [Sesi 04: AI Chat Assistant](#)

## Sesi 04: AI Chat Assistant

**Project 4 dari 5** — AI Chat Assistant dengan Mastra AI + Next.js

---

### ☐ Tujuan

- Setup framework Mastra AI untuk membangun AI agent
  - Membuat agent dengan tools (function calling)
  - Implementasi streaming response
  - Menambahkan RAG memory untuk konteks percakapan
  - Membangun frontend chat interface dengan Next.js
  - Deploy ke Vercel
- 

### ☐ Deliverable

- Live AI agent app: <https://ai-chat-namakamu.vercel.app>
  - GitHub repo: [github.com/namakamu/ai-chat-assistant](https://github.com/namakamu/ai-chat-assistant)
- 

### ☐ Tech Stack

Teknologi	Kegunaan
Mastra AI	Framework AI agent
OpenAI / Claude	LLM provider

Teknologi	Kegunaan
Next.js 14	Frontend + API routes
Tailwind CSS	Styling UI
Vercel AI SDK	Streaming chat
Upstash / Redis	RAG memory storage

## □ Langkah 1: Setup Project

```
npx create-mastra@latest ai-chat-assistant
cd ai-chat-assistant
```

*# Atau manual:*

```
mkdir ai-chat-assistant && cd ai-chat-assistant
npm init -y
npm install @mastra/core @mastra/memory @mastra/tts openai ai @ai-sdk/openai
npm install next react react-dom
npm install -D typescript @types/node @types/react tailwindcss
```

Struktur project:

```
ai-chat-assistant/
├── src/
│ ├── app/
│ │ ├── layout.tsx
│ │ ├── page.tsx
│ │ └── api/
│ │ └── chat/
│ │ └── route.ts
│ ├── agents/
│ │ └── assistant.ts
│ ├── tools/
│ │ ├── weather.ts
│ │ └── search.ts
│ ├── memory/
│ │ └── index.ts
│ └── lib/
│ └── mastra.ts
├── tailwind.config.ts
├── tsconfig.json
├── next.config.js
└── .env.local
```

## □ Langkah 2: Environment Variables

```
.env.local:

Pilih salah satu provider:
OPENAI_API_KEY="sk-..."
ATAU
ANTHROPIC_API_KEY="sk-ant-..."

Redis untuk memory (opsional)
UPSTASH_REDIS_REST_URL="https://..."
UPSTASH_REDIS_REST_TOKEN="..."
```

---

## □ Langkah 3: Setup Mastra Client

```
src/lib/mastra.ts:

import { Mastra } from '@mastra/core';
import { assistantAgent } from '../agents/assistant';

export const mastra = new Mastra({
 agents: { assistant: assistantAgent },
});
```

---

## □ Langkah 4: Tools

### Tool 1: Weather

```
src/tools/weather.ts:

import { createTool } from '@mastra/core';
import { z } from 'zod';

interface WeatherResponse {
 temperature: number;
 description: string;
 humidity: number;
 windSpeed: number;
 location: string;
}

export const weatherTool = createTool({
 id: 'get-weather',
 description: 'Dapatkan informasi cuaca untuk suatu kota',
```



```

inputSchema: z.object({
 location: z.string().describe('Nama kota atau lokasi'),
}),
outputSchema: z.object({
 temperature: z.number(),
 description: z.string(),
 humidity: z.number(),
 windSpeed: z.number(),
 location: z.string(),
}),
execute: async ({ context: { location } }): Promise<WeatherResponse> => {
 // Dalam production, panggil API cuaca sungguhan
 // Contoh: https://api.openweathermap.org/data/2.5/weather?q=${location}&app
 // Simulasi response untuk development
 const mockData: Record<string, WeatherResponse> = {
 jakarta: {
 temperature: 32,
 description: 'Cerah berawan',
 humidity: 78,
 windSpeed: 12,
 location: 'Jakarta, Indonesia',
 },
 bandung: {
 temperature: 24,
 description: 'Berawan',
 humidity: 65,
 windSpeed: 8,
 location: 'Bandung, Indonesia',
 },
 surabaya: {
 temperature: 33,
 description: 'Cerah',
 humidity: 70,
 windSpeed: 10,
 location: 'Surabaya, Indonesia',
 },
 'new york': {
 temperature: 18,
 description: 'Partly cloudy',
 humidity: 55,
 windSpeed: 15,
 location: 'New York, US',
 },
 tokyo: {
 temperature: 22,

```

```

 description: 'Light rain',
 humidity: 80,
 windSpeed: 10,
 location: 'Tokyo, Japan',
 },
];

 const key = location.toLowerCase().trim();
 const data = mockData[key];

 if (!data) {
 return {
 temperature: 28,
 description: 'Data tidak tersedia, menampilkan perkiraan default',
 humidity: 60,
 windSpeed: 5,
 location: location,
 };
 }

 return data;
 },
});

```

## Tool 2: Calculator

src/tools/calculator.ts:

```

import { createTool } from '@mastra/core';
import { z } from 'zod';

export const calculatorTool = createTool({
 id: 'calculator',
 description: 'Lakukan operasi matematika',
 inputSchema: z.object({
 expression: z.string().describe('Ekspresi matematika (contoh: 2 + 3 * 4)'),
 }),
 execute: async ({ context: { expression } }) => {
 // Hanya izinkan karakter yang aman
 const sanitized = expression.replace(/[^\0-9+\-*/.()]/g, '');
 try {
 // Gunakan Function constructor (AMAN karena sudah di-sanitize)
 const result = new Function(`return (${sanitized})`)();
 return {
 expression,
 result: String(result),
 };
 } catch (error) {
 return {
 expression,
 result: 'Error: Invalid expression',
 };
 }
 },
});

```

```

 };
 } catch {
 return {
 expression,
 result: 'Error: ekspresi tidak valid',
 };
 }
},
});

```

### Tool 3: Web Search (Simulasi)

src/tools/search.ts:

```

import { createTool } from '@mastra/core';
import { z } from 'zod';

export const searchTool = createTool({
 id: 'web-search',
 description: 'Cari informasi di web. Gunakan ketika user bertanya tentang berita',
 inputSchema: z.object({
 query: z.string().describe('Pertanyaan atau kata kunci pencarian'),
 }),
 execute: async ({ context: { query } }) => {
 // Dalam production, gunakan API search seperti SerpAPI, Tavily, atau Firecrawl
 // const result = await firecrawl.search(query);

 return {
 query,
 results: [
 {
 title: `${query} - Wikipedia`,
 snippet: `Informasi lengkap tentang ${query} dari ensiklopedia digital`,
 url: `https://id.wikipedia.org/wiki/${encodeURIComponent(query)}`,
 },
 {
 title: `Berita terbaru ${query}`,
 snippet: `Kumpulan berita dan artikel terbaru mengenai ${query}.`,
 url: `https://www.google.com/search?q=${encodeURIComponent(query)}`,
 },
],
 };
 },
});

```

## □ Langkah 5: Memory (RAG Context)

src/memory/index.ts:

```
import { Memory } from '@mastra/memory';

// In-memory storage untuk development
// Dalam production, ganti dengan memori persisten (Redis/PostgreSQL)
interface MemoryEntry {
 role: 'user' | 'assistant';
 content: string;
 timestamp: number;
}

class ChatMemory {
 private conversations: Map<string, MemoryEntry[]> = new Map();
 private maxEntries = 50; // Max entries per session

 async add(sessionId: string, entry: MemoryEntry): Promise<void> {
 if (!this.conversations.has(sessionId)) {
 this.conversations.set(sessionId, []);
 }
 const entries = this.conversations.get(sessionId)!;
 entries.push(entry);

 // Prune jika terlalu panjang
 if (entries.length > this.maxEntries) {
 this.conversations.set(sessionId, entries.slice(-this.maxEntries));
 }
 }

 async getContext(sessionId: string): Promise<string> {
 const entries = this.conversations.get(sessionId) || [];
 return entries
 .slice(-10) // Ambil 10 pesan terakhir untuk konteks
 .map((e) => `${e.role} === 'user' ? 'User' : 'Assistant': ${e.content}`)
 .join('\n');
 }

 async clear(sessionId: string): Promise<void> {
 this.conversations.delete(sessionId);
 }
}

export const chatMemory = new ChatMemory();
```

## □ Langkah 6: AI Agent

src/agents/assistant.ts:

```
import { Agent } from '@mastra/core';
import { openai } from '@ai-sdk/openai';
import { anthropic } from '@ai-sdk/anthropic';
import { weatherTool } from '../tools/weather';
import { calculatorTool } from '../tools/calculator';
import { searchTool } from '../tools/search';
import { chatMemory } from '../memory';
```

```
// Pilih provider:
// OpenAI: openai('gpt-4o')
// Claude: anthropic('claude-3-5-sonnet-20241022')
const model = openai('gpt-4o');
```

```
export const assistantAgent = new Agent({
 name: 'AI Assistant',
 instructions: `
```

Kamu adalah asisten AI yang membantu dan ramah. Kamu bisa:

1. Menjawab pertanyaan umum dengan akurat
2. Mencari informasi cuaca (gunakan tool get-weather)
3. Menghitung matematika (gunakan tool calculator)
4. Mencari informasi terbaru (gunakan tool web-search)
5. Mengingat konteks percakapan sebelumnya

Gunakan tools saat dibutuhkan. Jika tidak yakin, akui saja.  
Bahasa utama: Bahasa Indonesia. Jawab dengan jelas dan ringkas.

Ketika user bertanya tentang cuaca, SELALU gunakan tool get-weather.

Ketika user meminta perhitungan, SELALU gunakan tool calculator.

Ketika user bertanya tentang berita atau informasi terkini, gunakan tool web-sea

```
\n model,\n tools: [weatherTool, calculatorTool, searchTool],\n memory: {\n // Integrasi dengan memory system Mastra\n // Konfigurasi sesuai dokumentasi @mastra/memory\n },\n});
```

```
// Helper untuk chat dengan memory
```

```
export async function chatWithAgent(sessionId: string, message: string) {\n // Dapatkan konteks dari memory
```

```

const context = await chatMemory.getContext(sessionId);

// Kirim pesan ke agent
const response = await assistantAgent.generate([
 {
 role: 'system',
 content: `Context percakapan sebelumnya:\n${context}\n---`,
 },
 {
 role: 'user',
 content: message,
 },
]);

// Simpan ke memory
await chatMemory.add(sessionId, { role: 'user', content: message, timestamp: D
await chatMemory.add(sessionId, {
 role: 'assistant',
 content: response.text,
 timestamp: Date.now(),
});

return response;
}

```

---

## □ Langkah 7: API Route (Next.js)

src/app/api/chat/route.ts:

```

import { NextRequest } from 'next/server';
import { anthropic } from '@ai-sdk/anthropic';
import { openai } from '@ai-sdk/openai';
import { streamText, tool } from 'ai';
import { z } from 'zod';

// Tools untuk AI SDK
const weatherTool = tool({
 description: 'Dapatkan informasi cuaca untuk suatu kota',
 parameters: z.object({
 location: z.string().describe('Nama kota'),
 }),
 execute: async ({ location }) => {
 const mockData: Record<string, any> = {
 jakarta: { temperature: 32, description: 'Cerah berawan' },

```

```

 bandung: { temperature: 24, description: 'Berawan' },
 };
 return mockData[location.toLowerCase()] || { temperature: 28, description: ' ' };
 },
});

const calculatorTool = tool({
 description: 'Lakukan operasi matematika',
 parameters: z.object({
 expression: z.string().describe('Ekspresi matematika'),
 }),
 execute: async ({ expression }) => {
 const sanitized = expression.replace(/^[^0-9+\-*/.()]/g, '');
 try {
 const result = new Function(`return (${sanitized})`)();
 return { result: String(result) };
 } catch {
 return { error: 'Ekspresi tidak valid' };
 }
 },
});

// Pilih model
const model = openai('gpt-4o');

// Memory sederhana (in-memory)
const sessions = new Map<string, any[]>();

export async function POST(req: NextRequest) {
 const { messages, sessionId } = await req.json();

 // Simpan session untuk memory
 if (!sessions.has(sessionId)) {
 sessions.set(sessionId, []);
 }
 const history = sessions.get(sessionId)!;

 // Ambil 10 pesan terakhir dari history
 const contextMessages = history.slice(-10);

 const result = streamText({
 model,
 system: `Kamu asisten AI ramah berbahasa Indonesia. Gunakan tools jika perlu`,
 messages: [...contextMessages, ...messages],
 tools: {
 getWeather: weatherTool,
 },
 });
}

```

```

 calculate: calculatorTool,
 },
 });

 // Simpan pesan user ke history setelah response
 const lastUserMsg = messages[messages.length - 1];
 if (lastUserMsg?.role === 'user') {
 history.push(lastUserMsg);
 }

 // Streaming response
 return result.toDataStreamResponse({
 onFinish: async (event) => {
 // Simpan response assistant ke history
 history.push({
 role: 'assistant',
 content: event.text,
 });
 sessions.set(sessionId, history);
 },
 });
 }

 // Endpoint untuk clear session
 export async function DELETE(req: NextRequest) {
 const { sessionId } = await req.json();
 sessions.delete(sessionId);
 return Response.json({ success: true });
 }
}

```

---

## □ Langkah 8: Frontend Chat Interface

src/app/page.tsx:

```

'use client';

import { useState, useRef, useEffect } from 'react';
import { useChat } from 'ai/react';

interface Message {
 id: string;
 role: 'user' | 'assistant';
 content: string;
}

```



```

export default function ChatPage() {
 const [sessionId] = useState(() => crypto.randomUUID());
 const messagesEndRef = useRef<HTMLDivElement>(null);

 const { messages, input, handleInputChange, handleSubmit, isLoading, setMessages } = useChat({
 api: '/api/chat',
 body: { sessionId },
 onFinish: () => {
 // Scroll ke bawah setelah selesai
 setTimeout(() => {
 messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
 }, 100);
 },
 });

 // Auto-scroll saat pesan baru
 useEffect(() => {
 messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
 }, [messages]);

 const clearChat = async () => {
 await fetch('/api/chat', {
 method: 'DELETE',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ sessionId }),
 });
 setMessages([]);
 };

 return (
 <div className="flex flex-col h-screen bg-gray-50">
 { /* Header */ }
 <header className="bg-white shadow-sm border-b px-6 py-4">
 <div className="max-w-4xl mx-auto flex justify-between items-center">
 <div>
 <h1 className="text-xl font-bold text-gray-900">AI Chat Assistant</h1>
 <p className="text-sm text-gray-500">Tanya apa saja – AI siap membantu</p>
 </div>
 <button
 onClick={clearChat}
 className="px-4 py-2 text-sm border rounded-lg hover:bg-gray-50 transition">
 Hapus Percakapan
 </button>
 </div>
 </header>
 </div>
);
}

```

```
</header>
```

```
{/* Messages */}
```

```
<main className="flex-1 overflow-y-auto">
```

```
<div className="max-w-4xl mx-auto p-4 space-y-4">
```

```
{messages.length === 0 && (
```

```
<div className="text-center py-20">
```

```
<div className="text-6xl mb-4"></div>
```

```
<h2 className="text-2xl font-bold text-gray-900 mb-2">Halo! Ada ya
```

```
<p className="text-gray-500">
```

```
Coba tanyakan cuaca, hitung matematika, atau diskusi apa saja.
```

```
</p>
```

```
</div>
```

```
)}
```

```
{messages.map((m: Message) => (
```

```
<div key={m.id} className={`flex ${m.role === 'user' ? 'justify-end'}
```

```
<div
```

```
className={`max-w-[80%] rounded-2xl px-6 py-3 ${
```

```
m.role === 'user'
```

```
? 'bg-blue-600 text-white rounded-br-md'
```

```
: 'bg-white shadow-sm border rounded-bl-md'
```

```
}`}
```

```
>
```

```
<p className="text-sm whitespace-pre-wrap">{m.content}</p>
```

```
{m.role === 'assistant' && (
```

```
<div className="flex gap-2 mt-2 pt-2 border-t border-gray-100"
```

```
<button className="text-xs text-gray-400 hover:text-gray-600"
```

```
<button className="text-xs text-gray-400 hover:text-gray-600"
```

```
<button className="text-xs text-gray-400 hover:text-gray-600"
```

```
</div>
```

```
)}
```

```
</div>
```

```
</div>
```

```
))}
```

```
{isLoading && (
```

```
<div className="flex justify-start">
```

```
<div className="bg-white shadow-sm border rounded-2xl rounded-bl-m
```

```
<div className="flex gap-2">
```

```
<div className="w-2 h-2 bg-gray-400 rounded-full animate-bounc
```

```
<div className="w-2 h-2 bg-gray-400 rounded-full animate-bounc
```

```
<div className="w-2 h-2 bg-gray-400 rounded-full animate-bounc
```

```
</div>
```

```
</div>
```

```
</div>
```

```

 })

 <div ref={messagesEndRef} />
 </div>
</main>

{/* Input */}
<footer className="bg-white border-t px-6 py-4">
 <div className="max-w-4xl mx-auto">
 <form onSubmit={handleSubmit} className="flex gap-3">
 <input
 type="text"
 value={input}
 onChange={handleInputChange}
 placeholder="Ketik pesan..."
 disabled={isLoading}
 className="flex-1 px-5 py-3 border rounded-full focus:ring-2 focus-
 />
 <button
 type="submit"
 disabled={isLoading || !input.trim()}
 className="bg-blue-600 text-white px-6 py-3 rounded-full hover:bg-
 >
 Kirim
 </button>
 </form>
 <p className="text-xs text-gray-400 mt-2 text-center">
 AI dapat membuat kesalahan. Verifikasi informasi penting.
 </p>
 </div>
</footer>
</div>
);
}

```

## □ Langkah 9: Layout & Styling

src/app/layout.tsx:

```

import type { Metadata } from 'next';
import './globals.css';

export const metadata: Metadata = {
 title: 'AI Chat Assistant',

```

```

 description: 'AI Chat Assistant dengan Mastra AI + Next.js',
 };

export default function RootLayout({ children }: { children: React.ReactNode }) {
 return (
 <html lang="id">
 <body className="antialiased">{children}</body>
 </html>
);
}

src/app/globals.css:
@import "tailwindcss";

```

---

## □ Langkah 10: Konfigurasi

next.config.js:

```

/** @type {import('next').NextConfig} */
const nextConfig = {
 experimental: {
 serverComponentsExternalPackages: ['@mastra/core', '@mastra/memory'],
 },
};

```

module.exports = nextConfig;

tsconfig.json:

```

{
 "compilerOptions": {
 "target": "ES2017",
 "lib": ["dom", "dom.iterable", "esnext"],
 "allowJs": true,
 "skipLibCheck": true,
 "strict": true,
 "noEmit": true,
 "esModuleInterop": true,
 "module": "esnext",
 "moduleResolution": "bundler",
 "resolveJsonModule": true,
 "isolatedModules": true,
 "jsx": "preserve",
 "incremental": true,
 "plugins": [{ "name": "next" }],
 },
}

```

```
 "paths": { "@/*": ["./src/*"] }
 },
 "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
 "exclude": ["node_modules"]
}
```

---

## □ Langkah 11: Test Lokal

```
npm run dev
→ http://localhost:3000
```

Coba: 1. "Apa cuaca di Jakarta?" 2. "Hitung  $25 * 4 + 10$ " 3. "Siapa presiden pertama Indonesia?" 4. "Apa yang kita bicarakan sebelumnya?" (test memory)

Screenshot

Figure 8: Screenshot

## □ Langkah 12: Deploy ke Vercel

```
Install Vercel CLI
npm i -g vercel

Deploy
vercel

Atau hubungkan GitHub:
1. Push ke GitHub
2. Buka vercel.com → Import repo
3. Set environment variables:
- OPENAI_API_KEY
4. Deploy
```

Screenshot

Figure 9: Screenshot

## ▣ Pengembangan Lanjutan: RAG dengan Dokumen

Untuk membuat agent bisa membaca dokumen (PDF, website, dll):

```
npm install @mastra/rag @mastra/embeddings
```

```
src/memory/rag.ts:
```

```
import { RAG } from '@mastra/rag';

// Inisialisasi RAG dengan embedding
const rag = new RAG({
 embeddings: {
 provider: 'openai',
 model: 'text-embedding-3-small',
 },
 // Vector store bisa pakai:
 // - PostgreSQL (pgvector)
 // - Pinecone
 // - Supabase
 storage: {
 type: 'postgres',
 connectionString: process.env.DATABASE_URL!,
 },
});

// Index dokumen
export async function indexDocument(content: string, metadata?: any) {
 await rag.index({
 documents: [{ content, metadata }],
 });
}

// Search konteks relevan
export async function searchContext(query: string) {
 const results = await rag.search(query, { topK: 3 });
 return results.map((r) => r.content).join('\n\n');
}
```

---

## ▣ Latihan

1. **Multiple models** — tambahkan toggle antara GPT-4o dan Claude 3.5 Sonnet
2. **Voice input** — integrasikan Web Speech API untuk input suara
3. **Image analysis** — buat tool yang bisa menganalisa gambar menggunakan GPT-4 Vision

4. **Code execution** — tambahkan tool untuk menjalankan kode Python/JS di sandbox
  5. **Markdown rendering** — render kode block dengan syntax highlighting (react-markdown + rehype)
  6. **Persistent memory** — ganti in-memory dengan Redis/PostgreSQL
  7. **Chat history** — tampilkan list percakapan sebelumnya, bisa switch
  8. **Share chat** — fitur export/share percakapan sebagai link
- 

## □ Checklist

- ☐ Setup Mastra AI project
  - ☐ Weather tool berfungsi
  - ☐ Calculator tool berfungsi
  - ☐ Search tool berfungsi
  - ☐ Agent bisa memanggil tools sesuai konteks
  - ☐ Streaming response (typing effect)
  - ☐ Memory percakapan (konteks berlanjut)
  - ☐ Chat UI (input, bubble, scroll)
  - ☐ Responsive design
  - ☐ Deploy ke Vercel
  - ☐ API key terkonfigurasi di environment
- 

**Project 4 selesai!** Lanjut ke [Sesi 05: Production Deploy](#)

## Sesi 05: Production Deployment

**Project 5 dari 5** — Setup production-grade deployment dengan Docker, VPS, CI/CD, dan monitoring

---

## □ Tujuan

- Mengkonfigurasi aplikasi dengan Docker dan Docker Compose
- Menyewa dan mengkonfigurasi VPS (DigitalOcean / Linode / Contabo)
- Setup Nginx reverse proxy + SSL (Let's Encrypt / Cloudflare)
- Setup CI/CD dengan GitHub Actions
- Implementasi monitoring dengan Sentry
- Production checklist: health check, logging, backup, rate limiting, security headers

---

## ▢ Deliverable

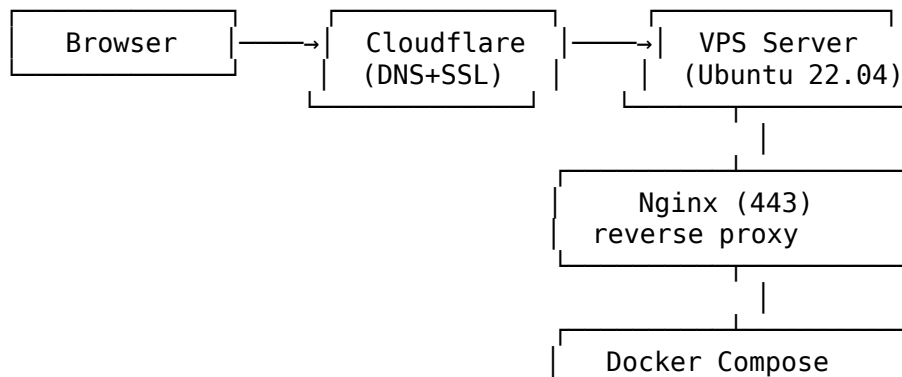
- Production-grade deployed app: <https://app.domainkamu.com>
  - GitHub Actions CI/CD pipeline
  - Monitoring dengan Sentry error tracking
  - Docker Compose config (app + DB + cache)
  - Backup & recovery script
- 

## ▢ Tech Stack

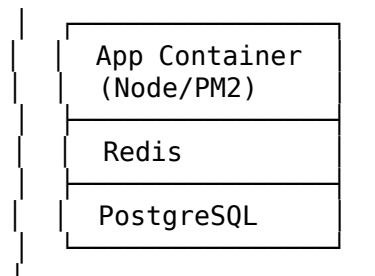
Teknologi	Kegunaan
Docker + Docker Compose	Containerization
VPS (Ubuntu 22.04)	Server
Nginx	Reverse proxy
Cloudflare	DNS + SSL + CDN
Let's Encrypt / Certbot	SSL certificate
PM2	Process manager (Node.js)
Redis	Cache & session store
PostgreSQL	Database
Sentry	Error monitoring
GitHub Actions	CI/CD pipeline
Uptime Kuma	Uptime monitoring
Grafana + Prometheus	Metrics (opsional)

---

## ▢ Arsitektur







---

## □ Langkah 1: Docker Setup

### Langkah 1.1: Dockerfile

Buat Dockerfile di root project:

*# Stage 1: Build*

**FROM** node:20-alpine **AS** builder

**WORKDIR** /app

*# Copy package files*

**COPY** package\*.json ./

**RUN** npm ci

*# Copy source*

**COPY** . .

*# Build*

**RUN** npm run build

*# Stage 2: Production*

**FROM** node:20-alpine **AS** production

**WORKDIR** /app

*# Install PM2*

**RUN** npm install -g pm2

*# Copy built artifacts*

**COPY** --from=builder /app/dist ./dist

**COPY** --from=builder /app/node\_modules ./node\_modules

**COPY** --from=builder /app/package.json ./

**COPY** --from=builder /app/prisma ./prisma

```

Expose port
EXPOSE 3000

Health check
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
 CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

Start with PM2
CMD ["pm2-runtime", "dist/index.js", "--name", "app", "-i", "max"]

```

## Langkah 1.2: Docker Compose

Buat docker-compose.yml:

```

version: '3.8'

services:
 app:
 build:
 context: .
 dockerfile: Dockerfile
 container_name: todo-app
 restart: unless-stopped
 depends_on:
 postgres:
 condition: service_healthy
 redis:
 condition: service_started
 environment:
 - NODE_ENV=production
 - DATABASE_URL=postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/${DB_NAME}
 - REDIS_URL=redis://redis:6379
 - JWT_SECRET=${JWT_SECRET}
 - FRONTEND_URL=${FRONTEND_URL}
 - SENTRY_DSN=${SENTRY_DSN}
 env_file:
 - .env.production
 ports:
 - "3000:3000"
 networks:
 - app-network
 volumes:
 - app-uploads:/app/uploads
 logging:
 driver: "json-file"
 options:

```

```

 max-size: "10m"
 max-file: "3"

postgres:
 image: postgres:16-alpine
 container_name: todo-postgres
 restart: unless-stopped
 environment:
 POSTGRES_USER: ${DB_USER}
 POSTGRES_PASSWORD: ${DB_PASSWORD}
 POSTGRES_DB: ${DB_NAME}
 volumes:
 - postgres-data:/var/lib/postgresql/data
 - ./backups:/backups
 networks:
 - app-network
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
 interval: 10s
 timeout: 5s
 retries: 5
 logging:
 driver: "json-file"
 options:
 max-size: "10m"
 max-file: "3"

redis:
 image: redis:7-alpine
 container_name: todo-redis
 restart: unless-stopped
 command: redis-server --appendonly yes --requirepass ${REDIS_PASSWORD}
 volumes:
 - redis-data:/data
 networks:
 - app-network
 logging:
 driver: "json-file"
 options:
 max-size: "10m"
 max-file: "3"

volumes:
 postgres-data:
 redis-data:
 app-uploads:

```

```
networks:
 app-network:
 driver: bridge
```

### **Langkah 1.3: .env.production**

```
Database
DB_USER=todo_user
DB_PASSWORD=generate-strong-password-here
DB_NAME=todo_production

JWT
JWT_SECRET=generate-strong-random-secret
JWT_EXPIRES_IN=7d

Redis
REDIS_PASSWORD=another-strong-password

URLs
FRONTEND_URL=https://app.domainkamu.com

Sentry
SENTRY_DSN=https://your-dsn@sentry.io/your-project

Rate Limit
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX=100
```

### **Langkah 1.4: .dockerignore**

```
node_modules
.git
.gitignore
*.md
.env
.env.local
.env.development
dist
coverage
test
tests
```

---

## □ Langkah 2: VPS Setup

### Langkah 2.1: Sewa VPS

Rekomendasi provider: - **Contabo** — Cloud VPS XS (\$6.99/bulan): 2 vCPU, 4GB RAM, 100GB SSD - **DigitalOcean** — Basic Droplet (\$6/bulan): 1 vCPU, 1GB RAM, 25GB SSD - **Linode** — Nanode 1GB (\$5/bulan): 1 vCPU, 1GB RAM, 25GB SSD

□ Contabo XS recommended untuk budget, cukup untuk app + DB + Redis

### Langkah 2.2: Initial Server Setup

SSH ke VPS:

```
ssh root@<server-ip>
```

Update & upgrade:

```
apt update && apt upgrade -y
```

Buat user non-root:

```
adduser deploy
usermod -aG sudo deploy
```

Setup SSH key:

```
Dari local machine
ssh-copy-id deploy@<server-ip>
```

### Langkah 2.3: Install Docker

```
Install Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

```
Install Docker Compose
sudo apt install -y docker-compose-plugin
```

```
Add user to docker group
sudo usermod -aG docker $USER
```

```
Test
docker --version
docker compose version
```

## Langkah 2.4: Firewall Setup

```
UFW
ufw allow OpenSSH
ufw allow 80/tcp
ufw allow 443/tcp
ufw allow 3000/tcp # Untuk testing saja, nanti dihapus
ufw enable
ufw status
```

## Langkah 2.5: Swap Memory (untuk VPS kecil)

```
fallocate -l 2G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab
```

---

## □ Langkah 3: Nginx & SSL

### Langkah 3.1: Install Nginx

```
apt install -y nginx
```

### Langkah 3.2: Nginx Config

Buat /etc/nginx/sites-available/todo-app:

```
server {
 listen 80;
 server_name app.domainkamu.com;

 # Redirect HTTP → HTTPS (setelah SSL terpasang)
 return 301 https://$server_name$request_uri;
}

server {
 listen 443 ssl http2;
 server_name app.domainkamu.com;

 # SSL (akan diisi Certbot nanti)
 ssl_certificate /etc/letsencrypt/live/app.domainkamu.com/fullchain.pem;
 ssl_certificate_key /etc/letsencrypt/live/app.domainkamu.com/privkey.pem;

 # Security headers
```

```

add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" a
add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" alw
add_header Content-Security-Policy "default-src 'self'; script-src 'self' 'u

Gzip
gzip on;
gzip_types text/plain text/css application/json application/javascript text/

Rate limiting
limit_req_zone $binary_remote_addr zone=api:10m rate=30r/s;
limit_req zone=api burst=50 nodelay;

location / {
 proxy_pass http://localhost:3000;
 proxy_http_version 1.1;
 proxy_set_header Upgrade $http_upgrade;
 proxy_set_header Connection 'upgrade';
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
 proxy_cache_bypass $http_upgrade;

 # Timeouts
 proxy_connect_timeout 60s;
 proxy_send_timeout 60s;
 proxy_read_timeout 60s;
}

Health check endpoint – no rate limit
location /health {
 proxy_pass http://localhost:3000/health;
 limit_req off;
 access_log off;
}

Static files cache
location ~* \.(jpg|jpeg|png|gif|ico|css|js|webp)$ {
 expires 30d;
 add_header Cache-Control "public, immutable";
}

```

```

Block access to sensitive files
location ~ /\.env {
 deny all;
}
location ~ /\.git {
 deny all;
}

Logs
access_log /var/log/nginx/todo-app-access.log;
error_log /var/log/nginx/todo-app-error.log;
}

```

Aktifkan site:

```

ln -s /etc/nginx/sites-available/todo-app /etc/nginx/sites-enabled/
nginx -t
systemctl reload nginx

```

### Langkah 3.3: SSL dengan Certbot

```
apt install -y certbot python3-certbot-nginx
```

*# Dapatkan SSL certificate*

```
certbot --nginx -d app.domainkamu.com --non-interactive --agree-tos -m email@dom
```

*# Test auto-renewal*

```
certbot renew --dry-run
```

### Langkah 3.4: Setup dengan Cloudflare (Alternatif)

1. Domain → Cloudflare → DNS → tambah A record ke IP VPS (proxy: enabled)
2. Cloudflare SSL/TLS → Full (Strict)
3. Cloudflare handle SSL secara otomatis
4. Di Nginx, cukup listen 80 dan trust Cloudflare

```

server {
 listen 80;
 server_name app.domainkamu.com;

 # Trust Cloudflare IPs
 set_real_ip_from 103.21.244.0/22;
 set_real_ip_from 103.22.200.0/22;
 # ... (daftar lengkap IP Cloudflare)
 real_ip_header CF-Connecting-IP;
}

```



```
 location / {
 proxy_pass http://localhost:3000;
 ...
 }
}
```

---

## □ Langkah 4: Deploy App

### Langkah 4.1: Clone & Setup

```
cd /home/deploy
git clone https://github.com/namakamu/todo-app.git
cd todo-app
```

```
Buat .env.production
nano .env.production
Paste environment variables
```

```
Build & start
docker compose build
docker compose up -d
```

```
Check logs
docker compose logs -f app
```

### Langkah 4.2: Prisma Migration

```
docker compose exec app npx prisma migrate deploy
```

### Langkah 4.3: Verify

```
curl http://localhost:3000/health
→ {"status":"OK","timestamp":"2025-01-01T00:00:00.000Z"}
```

```
curl http://localhost:3000/api/books?limit=1
→ {"success":true,"data":[...],"meta":{"..."}}
```

Screenshot

Figure 10: Screenshot

---

## □ Langkah 5: CI/CD dengan GitHub Actions

### Langkah 5.1: GitHub Secrets

Tambahkan di GitHub repo → Settings → Secrets and variables → Actions:

Secret	Value
DOCKER_USERNAME	Docker Hub username
DOCKER_PASSWORD	Docker Hub password/token
SERVER_HOST	VPS IP address
SERVER_USERNAME	deploy
SSH_PRIVATE_KEY	Private key untuk SSH ke VPS
ENV_PRODUCTION	Isi file .env.production (base64 encoded)

### Langkah 5.2: GitHub Actions Workflow

Buat .github/workflows/deploy.yml:

```
name: Deploy to Production

on:
 push:
 branches: [main]

jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4

 - uses: actions/setup-node@v4
 with:
 node-version: 20

 - name: Install dependencies
 run: npm ci

 - name: Lint
 run: npm run lint

 - name: Type check
 run: npm run typecheck

 - name: Run tests
 run: npm test
```

```

build-and-deploy:
 needs: test
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4

 - name: Set up Docker Buildx
 uses: docker/setup-buildx-action@v3

 - name: Login to Docker Hub
 uses: docker/login-action@v3
 with:
 username: ${ secrets.DOCKER_USERNAME }
 password: ${ secrets.DOCKER_PASSWORD }

 - name: Build and push Docker image
 uses: docker/build-push-action@v6
 with:
 context: .
 push: true
 tags: |
 ${ secrets.DOCKER_USERNAME }/todo-app:latest
 ${ secrets.DOCKER_USERNAME }/todo-app:${ github.sha }
 cache-from: type=registry,ref=${ secrets.DOCKER_USERNAME }/todo-app:
 cache-to: type=inline

 - name: Deploy to VPS
 uses: appleboy/ssh-action@v1.0.3
 with:
 host: ${ secrets.SERVER_HOST }
 username: ${ secrets.SERVER_USERNAME }
 key: ${ secrets.SSH_PRIVATE_KEY }
 script: |
 cd /home/deploy/todo-app
 docker compose pull app
 docker compose up -d --force-recreate app
 docker image prune -f

 - name: Health check
 run: |
 sleep 15
 curl -f https://app.domainkamu.com/health || exit 1

```

---

## □ Langkah 6: Monitoring & Error Tracking

### Langkah 6.1: Sentry Setup

Backend — install Sentry:

```
npm install @sentry/node @sentry/profiling-node
```

src/config/sentry.ts:

```
import * as Sentry from '@sentry/node';
import { env } from './env';

export function initSentry() {
 if (!env.sentryDsn) return;

 Sentry.init({
 dsn: env.sentryDsn,
 environment: env.nodeEnv,
 tracesSampleRate: env.nodeEnv === 'production' ? 0.1 : 1.0,
 profilesSampleRate: 0.1,
 integrations: [
 new Sentry.Integrations.Http({ tracing: true }),
 new Sentry.Integrations.Express({ app: true }),
],
 });
}
```

Tambahkan ke app.ts:

```
import { initSentry } from './config/sentry';

initSentry();

// Sentry handler harus paling awal
app.use(Sentry.Handlers.requestHandler());
app.use(Sentry.Handlers.tracingHandler());

// ... routes ...

// Error handler – Sentry
app.use(Sentry.Handlers.errorHandler());
```

### Langkah 6.2: Uptime Monitoring

Install **Uptime Kuma** di VPS terpisah atau Docker:

```
docker run -d \
 --name uptime-kuma \
```

```
-p 3001:3001 \
-v uptime-kuma-data:/app/data \
--restart unless-stopped \
louislam/uptime-kuma:latest
```

Buka `http://<vps-ip>:3001`, buat monitor untuk `https://app.domainkamu.com/health`.

### Langkah 6.3: Logging

Logging dengan Winston + daily rotate:

```
npm install winston winston-daily-rotate-file
```

```
src/lib/logger.ts:
```

```
import winston from 'winston';
import DailyRotateFile from 'winston-daily-rotate-file';

const logger = winston.createLogger({
 level: process.env.NODE_ENV === 'production' ? 'info' : 'debug',
 format: winston.format.combine(
 winston.format.timestamp(),
 winston.format.errors({ stack: true }),
 winston.format.json()
),
 defaultMeta: { service: 'todo-app' },
 transports: [
 new DailyRotateFile({
 filename: 'logs/error-%DATE%.log',
 datePattern: 'YYYY-MM-DD',
 level: 'error',
 maxFiles: '30d',
 }),
 new DailyRotateFile({
 filename: 'logs/combined-%DATE%.log',
 datePattern: 'YYYY-MM-DD',
 maxFiles: '14d',
 }),
],
});

if (process.env.NODE_ENV !== 'production') {
 logger.add(new winston.transports.Console({
 format: winston.format.combine(
 winston.format.colorize(),
 winston.format.simple()
),
 }));
}
```

```
 }));
}
```

```
export default logger;
```

---

## □ Langkah 7: Backup & Disaster Recovery

### Langkah 7.1: Database Backup Script

Buat scripts/backup.sh:

```
#!/bin/bash
set -e
```

```
BACKUP_DIR="/backups"
DB_NAME="todo_production"
DB_USER="todo_user"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/${DB_NAME}_${TIMESTAMP}.sql.gz"
RETENTION_DAYS=7
```

```
Backup PostgreSQL
```

```
docker compose exec -T postgres pg_dump -U ${DB_USER} ${DB_NAME} | gzip > ${BACKUP_FILE}
```

```
Backup Redis
```

```
docker compose exec -T redis redis-cli -a ${REDIS_PASSWORD} save
```

```
docker compose cp redis:/data/dump.rdb ${BACKUP_DIR}/redis_${TIMESTAMP}.rdb
```

```
Hapus backup lebih dari RETENTION_DAYS
```

```
find ${BACKUP_DIR} -name "${DB_NAME}_*.sql.gz" -mtime +${RETENTION_DAYS} -delete
```

```
find ${BACKUP_DIR} -name "redis_*.rdb" -mtime +${RETENTION_DAYS} -delete
```

```
echo "Backup selesai: ${BACKUP_FILE}"
```

Jadwalkan dengan cron:

```
chmod +x scripts/backup.sh
```

```
crontab -e
```

```
Tambahkan: 0 3 * * * /home/deploy/todo-app/scripts/backup.sh
```

### Langkah 7.2: Restore Script

scripts/restore.sh:

```
#!/bin/bash
set -e
```

```

BACKUP_FILE=$1
if [-z "$BACKUP_FILE"]; then
 echo "Usage: $0 <backup-file.sql.gz>"
 exit 1
fi

gunzip -c ${BACKUP_FILE} | docker compose exec -T postgres psql -U ${DB_USER} ${DB_NAME} <>
echo "Restore selesai dari: ${BACKUP_FILE}"

```

## ❑ Langkah 8: Production Checklist

### ❑ Security

Item	Status	Keterangan
SSH key-only auth	<input type="checkbox"/>	Nonaktifkan password SSH
UFW firewall	<input type="checkbox"/>	Hanya buka 22, 80, 443
Fail2ban	<input type="checkbox"/>	Proteksi brute force SSH
Auto security updates	<input type="checkbox"/>	unattended-upgrades
HTTPS (SSL)	<input type="checkbox"/>	Cloudflare atau Certbot
Security headers	<input type="checkbox"/>	Nginx config
Rate limiting	<input type="checkbox"/>	Nginx limit_req
CORS	<input type="checkbox"/>	Batasi origin di backend
Environment secrets	<input type="checkbox"/>	Jangan commit .env
Docker non-root	<input type="checkbox"/>	Jangan jalankan container sebagai root

### ❑ Reliability

Item	Status	Keterangan
Health endpoint	<input type="checkbox"/>	/health
Auto-restart	<input type="checkbox"/>	Docker restart: unless-stopped
Resource limits	<input type="checkbox"/>	Docker mem_limit, cpus
Graceful shutdown	<input type="checkbox"/>	SIGTERM handler di app
Database connection pool	<input type="checkbox"/>	Prisma connection_limit

### ❑ Data

Item	Status	Keterangan
Automated DB backup	<input type="checkbox"/>	Script cron harian
Off-site backup	<input type="checkbox"/>	Upload ke S3/Backblaze
Backup retention	<input type="checkbox"/>	Minimal 7 hari
Encrypted backups	<input type="checkbox"/>	GPG encrypt

## □ Monitoring

Item	Status	Keterangan
Error tracking (Sentry)	<input type="checkbox"/>	Production errors
Uptime monitoring	<input type="checkbox"/>	Uptime Kuma
Server monitoring	<input type="checkbox"/>	htop, docker stats
Log rotation	<input type="checkbox"/>	Winston daily rotate
Alert (email/telegram)	<input type="checkbox"/>	Sentry + Uptime Kuma alerts

## □ Performance

Item	Status	Keterangan
Nginx gzip	<input type="checkbox"/>	Kompresi response
Static file caching	<input type="checkbox"/>	Cache headers
Redis cache	<input type="checkbox"/>	Cache query results
Database indexing	<input type="checkbox"/>	Index kolom yang sering di-query
Load testing	<input type="checkbox"/>	k6 atau autocannon

## □ CI/CD

Item	Status	Keterangan
Tests on push	<input type="checkbox"/>	GitHub Actions
Auto lint	<input type="checkbox"/>	ESLint
Type check	<input type="checkbox"/>	TypeScript strict
Docker build	<input type="checkbox"/>	CI build
Deploy on main	<input type="checkbox"/>	Auto deploy
Rollback plan	<input type="checkbox"/>	Deploy ulang tag sebelumnya

## □ Langkah 9: Load Testing (Opsional)

Install k6 dan test:



```
Install k6
sudo apt install -y k6

Test file: loadtest.js
cat > loadtest.js << 'EOF'
import http from 'k6/http';
import { sleep, check } from 'k6';

export const options = {
 stages: [
 { duration: '1m', target: 50 },
 { duration: '2m', target: 100 },
 { duration: '1m', target: 0 },
],
};

export default function () {
 const res = http.get('https://app.domainkamu.com/health');
 check(res, { 'status 200': (r) => r.status === 200 });
 sleep(1);
}
EOF

k6 run loadtest.js
```

---

## □ Langkah 10: Rollback Plan

Jika deploy bermasalah:

```
Di VPS
cd /home/deploy/todo-app

Rollback ke tag sebelumnya
docker compose stop app
docker compose up -d app # atau pull versi sebelumnya

Atau rollback docker image
docker compose down
git checkout <previous-tag>
docker compose build
docker compose up -d
```

Untuk rollback cepat di GitHub Actions:

```
Tambahkan workflow manual rollback
```

```

name: Rollback
on:
 workflow_dispatch:
 inputs:
 tag:
 description: 'Image tag to deploy'
 required: true

jobs:
 deploy:
 runs-on: ubuntu-latest
 steps:
 - name: Deploy specific tag
 uses: appleboy/ssh-action@v1.0.3
 with:
 host: ${ secrets.SERVER_HOST }
 username: ${ secrets.SERVER_USERNAME }
 key: ${ secrets.SSH_PRIVATE_KEY }
 script: |
 cd /home/deploy/todo-app
 TAG=${ github.event.inputs.tag }
 sed -i "s|image:.*|image: ${ secrets.DOCKER_USERNAME }/todo-app:${ TAG }|"
 docker compose up -d app

```

---

## □ Latihan

1. **Tambah monitoring Grafana + Prometheus** — kumpulkan metrics container dengan cAdvisor
  2. **Setup staging environment** — buat docker-compose.staging.yml dengan database terpisah
  3. **Implementasi rate limiting di backend** — tambahkan express-rate-limit dengan Redis store
  4. **Database connection pooling** — konfigurasi Prisma connection pool untuk production
  5. **SSH hardening** — implementasikan 2FA, port knocking, atau Wireguard tunnel
  6. **S3 file uploads** — ganti local file storage dengan S3/MinIO untuk file uploads
  7. **Webhook alerts** — kirim notifikasi ke Telegram/Discord ketika backup gagal
  8. **Zero-downtime deployment** — implementasikan blue-green deployment dengan load balancer
-

## □ Final Checklist

- ☐ Dockerfile multi-stage build
- ☐ Docker Compose (app + PostgreSQL + Redis)
- ☐ VPS: SSH, Docker, Nginx installed
- ☐ Firewall configured (UFW)
- ☐ Domain pointed to VPS (Cloudflare)
- ☐ SSL certificate (Let's Encrypt / Cloudflare)
- ☐ Nginx reverse proxy with security headers
- ☐ App running (docker compose up)
- ☐ Prisma migration executed
- ☐ Health check passes
- ☐ GitHub Actions: test → build → deploy
- ☐ Sentry error tracking active
- ☐ Uptime monitoring active
- ☐ Database backup script + cron
- ☐ Rate limiting configured
- ☐ Production checklist all checked

---

## □ Selamat! Kamu sudah menyelesaikan semua 5 project portfolio!

Kamu sekarang memiliki: 1. Landing page portfolio sendiri (Vercel/Netlify) 2. REST API production-ready dengan dokumentasi (Railway) 3. Fullstack Todo App dengan JWT + OAuth (Railway + Vercel) 4. AI Chat Assistant dengan Mastra AI (Vercel) 5. Production-grade deployment dengan Docker + VPS + CI/CD

**5 project nyata = portfolio yang kuat untuk melamar kerja sebagai fullstack developer.**

Jangan lupa update landing page project 1 dengan link ke semua project lainnya!

Kembali ke [README](#)